Temporal Frictions and Judicial Outcomes:

Analyzing the Impact of Time Delays on Criminal Sentencing in Cook County (2020–2024)

Tian Tong, Georgetown University

In [48]:
```
import random

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import requests

from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

from pandas.plotting import parallel_coordinates
```

In [5]:
```
# Load the data
df = pd.read_csv("Data/Sentencing_20241028.csv")
df.head()
```

```
/var/folders/jt/3wdp6b7d0mj145sqgh8_fd480000gn/T/ipykernel_30268/1749622318.
py:2: DtypeWarning: Columns (10,11,14,25) have mixed types. Specify dtype op
tion on import or set low_memory=False.
  df = pd.read_csv("/Users/tyf981126/Desktop/Criminal_Sentencing_Delay_Analy
sis/Data/Sentencing_20241028.csv")
```

Out[5]:

	CASE_ID	CASE_PARTICIPANT_ID	RECEIVED_DATE	OFFENSE_CATEGORY	PR
0	149,352,426,337	414,373,090,914	08/15/1984 12:00:00 AM	PROMIS Conversion	
1	149,352,426,337	414,373,090,914	08/15/1984 12:00:00 AM	PROMIS Conversion	
2	149,352,426,337	414,373,090,914	08/15/1984 12:00:00 AM	PROMIS Conversion	
3	149,352,426,337	414,373,090,914	08/15/1984 12:00:00 AM	PROMIS Conversion	
4	149,352,426,337	414,373,090,914	08/15/1984 12:00:00 AM	PROMIS Conversion	

5 rows × 41 columns

In [6]:
```
df.shape
```

Out[6]:
```
(303963, 41)
```

In [7]:
```
print(df.columns)
```

```
Index(['CASE_ID', 'CASE_PARTICIPANT_ID', 'RECEIVED_DATE', 'OFFENSE_CATEGOR
Y',
       'PRIMARY_CHARGE_FLAG', 'CHARGE_ID', 'CHARGE_VERSION_ID',
       'DISPOSITION_CHARGED_OFFENSE_TITLE', 'CHARGE_COUNT', 'DISPOSITION_DAT
E',
       'DISPOSITION_CHARGED_CHAPTER', 'DISPOSITION_CHARGED_ACT',
       'DISPOSITION_CHARGED_SECTION', 'DISPOSITION_CHARGED_CLASS',
       'DISPOSITION_CHARGED_AOIC', 'CHARGE_DISPOSITION',
       'CHARGE_DISPOSITION_REASON', 'SENTENCE_JUDGE', 'SENTENCE_COURT_NAME',
       'SENTENCE_COURT_FACILITY', 'SENTENCE_PHASE', 'SENTENCE_DATE',
       'SENTENCE_TYPE', 'CURRENT_SENTENCE_FLAG', 'COMMITMENT_TYPE',
       'COMMITMENT_TERM', 'COMMITMENT_UNIT', 'LENGTH_OF_CASE_in_Days',
       'AGE_AT_INCIDENT', 'RACE', 'GENDER', 'INCIDENT_CITY',
       'INCIDENT_BEGIN_DATE', 'INCIDENT_END_DATE', 'LAW_ENFORCEMENT_AGENCY',
       'LAW_ENFORCEMENT_UNIT', 'ARREST_DATE', 'FELONY_REVIEW_DATE',
       'FELONY_REVIEW_RESULT', 'ARRAIGNMENT_DATE', 'UPDATED_OFFENSE_CATEGOR
Y'],
      dtype='object')
```

Change format of dates

In [8]:
```
# Convert date columns to datetime format
date_format = '%m/%d/%Y %I:%M:%S %p'
date_columns = ['INCIDENT_BEGIN_DATE', 'INCIDENT_END_DATE', 'ARREST_DATE', '

for col in date_columns:
    df[col] = pd.to_datetime(df[col], format=date_format, errors='coerce')
```

Make sure each date is in correct range(not exceeding current year of 2024)

In [9]:

```python
# Define our date boundaries
start_date = pd.to_datetime('2020-01-01')
end_date = pd.to_datetime('2024-10-28')

# Create mask for valid dates
mask = (
    # Time period constraints
    (df['INCIDENT_BEGIN_DATE'] >= start_date) &
    (df['INCIDENT_BEGIN_DATE'] <= end_date) &
    (df['SENTENCE_DATE'] <= end_date) &

    # Logical sequence constraints
    (df['ARREST_DATE'] >= df['INCIDENT_BEGIN_DATE']) &
    (df['SENTENCE_DATE'] >= df['ARREST_DATE']) &

    # Ensure dates exist
    (df['INCIDENT_BEGIN_DATE'].notna()) &
    (df['ARREST_DATE'].notna()) &
    (df['SENTENCE_DATE'].notna())
)

# Apply the filter
df_filtered = df[mask].copy()

# Print summary of filtering results
print("\nFiltering results:")
print(f"Original records: {len(df):,}")
print(f"Filtered records: {len(df_filtered):,}")
print(f"Percentage kept: {(len(df_filtered)/len(df))*100:.2f}%")

# Check date ranges after filtering
print("\nDate ranges after filtering:")
for col in date_columns:
    print(f"\n{col}:")
    print(f"Minimum: {df_filtered[col].min()}")
    print(f"Maximum: {df_filtered[col].max()}")
```

```
Filtering results:
Original records: 303,963
Filtered records: 43,205
Percentage kept: 14.21%

Date ranges after filtering:

INCIDENT_BEGIN_DATE:
Minimum: 2020-01-01 00:00:00
Maximum: 2024-09-14 00:00:00

INCIDENT_END_DATE:
Minimum: 2020-01-01 00:00:00
Maximum: 2024-07-01 00:00:00

ARREST_DATE:
Minimum: 2020-01-01 00:19:00
Maximum: 2024-09-14 21:52:00

SENTENCE_DATE:
Minimum: 2020-01-22 00:00:00
Maximum: 2024-10-24 00:00:00
```

Adding new Variables

In [10]:

```python
def standardize_sentence_length(term, unit):

    if pd.isna(term) or pd.isna(unit):
        return None

    try:
        # Handle special cases
        if unit == 'Natural Life':
            return 1440  # 120 years * 12 months

        term = float(term)
        if term < 0:  # Check for negative terms
            return None

        # Standard conversions
        conversion_dict = {
            'Year(s)': term * 12,
            'Months': term,
            'Days': term / 30.44  # Using average month length
        }

        standardized_term = conversion_dict.get(unit, None)

        # Drop values greater than 1440 months
        if standardized_term and standardized_term > 1440:
            return None

        return standardized_term
```

```
    except (ValueError, TypeError):
        return None
```

In [11]:
```
# Define the time delay between crime and arrest
df_filtered['time_delay_days'] = (
    df_filtered['ARREST_DATE'] - df_filtered['INCIDENT_BEGIN_DATE']
).dt.days

# Create both numeric and categorical versions
df_filtered['covid_period_num'] = pd.cut(
    df_filtered['INCIDENT_BEGIN_DATE'],
    bins=[
        pd.to_datetime('2019-12-31'),
        pd.to_datetime('2020-03-15'),
        pd.to_datetime('2022-01-01'),
        pd.to_datetime('2024-10-28')
    ],
    labels=[1, 2, 3]
)

df_filtered['covid_period_cat'] = pd.cut(
    df_filtered['INCIDENT_BEGIN_DATE'],
    bins=[
        pd.to_datetime('2019-12-31'),
        pd.to_datetime('2020-03-15'),
        pd.to_datetime('2022-01-01'),
        pd.to_datetime('2024-10-28')
    ],
    labels=['pre_covid', 'peak_covid', 'post_covid']
)

# Length of sentence months
df_filtered['sentence_months'] = df_filtered.apply(
    lambda x: standardize_sentence_length(x['COMMITMENT_TERM'], x['COMMITMEN
    axis=1
)
```

Statiscial summary of time_delay

In [12]:
```
# Validate statistics
print("New time delay statistics (days):")
print(df_filtered['time_delay_days'].describe())
```

```
New time delay statistics (days):
count    43205.000000
mean        17.092142
std         69.689557
min          0.000000
25%          0.000000
50%          0.000000
75%          0.000000
max       1223.000000
Name: time_delay_days, dtype: float64
```

In [13]:
```
# Check the distribution of delays among all
no_delay_count = (df_filtered['time_delay_days'] == 0).sum()
total_count = len(df_filtered)
print(f"Proportion of cases with no delay: {no_delay_count / total_count:.2%
```

```
Proportion of cases with no delay: 78.28%
```

- A large number of zero-delay cases (at least 75% of data points are 0)
- Some extreme delays (up to 1223 days)
- A mean of 17 days with a large standard deviation (70 days)

This distribution pattern suggests several important considerations:

- Statistical Concerns:The highly skewed distribution could affect model performance
- Zero-inflation might lead to biased results
- The relationship between delay and severity might be masked by the dominance of zero-delay cases

Check for missing values

In [14]:
```
# Get missing value counts and percentages for each column
missing_info = pd.DataFrame({
    'Missing Count': df_filtered.isnull().sum(),
    'Missing Percentage': (df_filtered.isnull().sum() / len(df_filtered) * 1
})

# Sort by missing percentage in descending order
missing_info = missing_info.sort_values('Missing Count', ascending=False)

# Display columns with missing values
print("Columns with missing values:")
print(missing_info[missing_info['Missing Count'] > 0])

total_columns = len(df_filtered.columns)
columns_with_missing = len(missing_info[missing_info['Missing Count'] > 0])

print(f"\nTotal number of columns: {total_columns}")
print(f"Number of columns with missing values: {columns_with_missing}")

key_vars = ['time_delay_days', 'sentence_months', 'covid_period_num', 'covic
print("\nMissing values in key variables:")
for var in key_vars:
    missing = df_filtered[var].isnull().sum()
    print(f"{var}: {missing} missing values ({(missing/len(df_filtered)*100)
```

```
Columns with missing values:
                           Missing Count  Missing Percentage
CHARGE_DISPOSITION_REASON          43103               99.76
INCIDENT_END_DATE                  41705               96.53
LAW_ENFORCEMENT_UNIT               36340               84.11
FELONY_REVIEW_RESULT                6935               16.05
FELONY_REVIEW_DATE                  6935               16.05
INCIDENT_CITY                        803                1.86
LENGTH_OF_CASE_in_Days               762                1.76
ARRAIGNMENT_DATE                     762                1.76
RACE                                 333                0.77
SENTENCE_COURT_FACILITY              283                0.66
GENDER                               204                0.47
sentence_months                      165                0.38
COMMITMENT_TYPE                       80                0.19
COMMITMENT_UNIT                       80                0.19
COMMITMENT_TERM                       80                0.19
AGE_AT_INCIDENT                       33                0.08
SENTENCE_COURT_NAME                    3                0.01

Total number of columns: 45
Number of columns with missing values: 17

Missing values in key variables:
time_delay_days: 0 missing values (0.00%)
sentence_months: 165 missing values (0.38%)
covid_period_num: 0 missing values (0.00%)
covid_period_cat: 0 missing values (0.00%)
```

In [15]:

```python
# List of columns we want to check for missing values
columns_to_check = [
    'LENGTH_OF_CASE_in_Days',
    'RACE',
    'GENDER',
    'sentence_months',
    'COMMITMENT_UNIT',
    'COMMITMENT_TYPE',
    'COMMITMENT_TERM',
    'AGE_AT_INCIDENT'
]

# Remove rows with missing values in these columns
df_clean = df_filtered.dropna(subset=columns_to_check)

print(f"Original number of records: {len(df_filtered)}")
print(f"Number of records after removing missing values: {len(df_clean)}")
print(f"Percentage of data retained: {(len(df_clean)/len(df_filtered)*100):.

missing_after = df_clean[columns_to_check].isnull().sum()
print("\nMissing values after cleaning:")
print(missing_after[missing_after > 0])
```

```
Original number of records: 43205
Number of records after removing missing values: 41986
Percentage of data retained: 97.18%

Missing values after cleaning:
Series([], dtype: int64)
```

Create the target variable: Punishment Level

Construct target: map commitment/sentence type to an ordinal severity level (1=supervision/light, 2=probation, 3=incarceration/commitment), assign specific value to the and use it to compute the punishment outcome.

In [17]:
```python
severity_mapping = {
    'Illinois Department of Corrections': 3,
    'Cook County Department of Corrections': 3,
    'Juvenile IDOC': 3,
    'Home Confinement': 3,
    'Cook County Boot Camp': 3,
    'Probation': 2,
    'Mental Health Probation': 2,
    'Drug Court Probation': 2,
    '710/410 Probation': 2,
    'Sex Offender Probation': 2,
    'Repeat Offender Probation': 2,
    'Conditional Discharge': 1,
    'Court Supervision': 1,
    'Conditional Release': 1,
    'Drug School': 1,
    'Vocational Rehabilitation Impact Center(VRIC)': 1
}

df_filtered['severity_level'] = df_filtered['COMMITMENT_TYPE'].map(severity_

df_filtered['weighted_punishment'] = df_filtered['sentence_months'] * df_fil
```

In [18]:
```python
# Apply alternative weights
alternative_weights = {
    3: 9,
    2: 4,
    1: 1
}

df_filtered['severity_level_alt'] = df_filtered['severity_level'].map(alterr
df_filtered['weighted_punishment_alt'] = (
    df_filtered['sentence_months'] * df_filtered['severity_level_alt']
)

# Inspect statistics of the alternative weighting
print("\nWeighted Punishment (Alternative) Statistics:")
print(df_filtered['weighted_punishment_alt'].describe())

# Correlation with original and alternative weighting
```

```
print("\nCorrelation between weighted and alternative weighted punishment:")
print(df_filtered[['weighted_punishment', 'weighted_punishment_alt']].corr()
```

```
Weighted Punishment (Alternative) Statistics:
count    42869.000000
mean       232.701807
std        439.679268
min          0.000000
25%         96.000000
50%         96.000000
75%        288.000000
max      12960.000000
Name: weighted_punishment_alt, dtype: float64

Correlation between weighted and alternative weighted punishment:
                         weighted_punishment  weighted_punishment_alt
weighted_punishment                 1.000000                 0.996808
weighted_punishment_alt             0.996808                 1.000000
```

In [19]:
```
# Check distribution of severity levels
print("\nSeverity Level Distribution:")
print(df_filtered['severity_level'].value_counts(normalize=True))

# Check weighted punishment statistics
print("\nWeighted Punishment Statistics:")
print(df_filtered['weighted_punishment'].describe())

# Compare with unweighted sentences
print("\nCorrelation between weighted and unweighted punishment:")
print(df_filtered[['sentence_months', 'weighted_punishment']].corr())
```

```
Severity Level Distribution:
severity_level
3.0    0.510815
2.0    0.450909
1.0    0.038276
Name: proportion, dtype: float64

Weighted Punishment Statistics:
count    42869.000000
mean        84.869732
std        145.687663
min          0.000000
25%         36.000000
50%         48.000000
75%         99.000000
max       4320.000000
Name: weighted_punishment, dtype: float64

Correlation between weighted and unweighted punishment:
                     sentence_months  weighted_punishment
sentence_months                1.000                0.992
weighted_punishment            0.992                1.000
```

In [20]:
```
df_filtered['weighted_punishment_std'] = (
    df_filtered['weighted_punishment'] - df_filtered['weighted_punishment'].
) / df_filtered['weighted_punishment'].std()
```

```
df_filtered['weighted_punishment_alt_std'] = (
    df_filtered['weighted_punishment_alt'] - df_filtered['weighted_punishmer
) / df_filtered['weighted_punishment_alt'].std()

print(df_filtered[['weighted_punishment', 'weighted_punishment_std', 'weight
```

```
       weighted_punishment  weighted_punishment_std  weighted_punishment_alt
\
count         42869.000000             4.286900e+04             42869.000000
mean             84.869732             8.221073e-17               232.701807
std             145.687663             1.000000e+00               439.679268
min               0.000000            -5.825458e-01                 0.000000
25%              36.000000            -3.354418e-01                96.000000
50%              48.000000            -2.530738e-01                96.000000
75%              99.000000             9.699015e-02               288.000000
max            4320.000000             2.906993e+01             12960.000000

       weighted_punishment_alt_std
count                 4.286900e+04
mean                  5.834310e-17
std                   1.000000e+00
min                  -5.292535e-01
25%                  -3.109126e-01
50%                  -3.109126e-01
75%                   1.257694e-01
max                   2.894678e+01
```

- Use the raw variables (weighted_punishment and weighted_punishment_alt) for:Exploratory analysis/Descriptive statistics/Reporting or presentations to non-technical audiences.
- Use the standardized versions (weighted_punishment_std and weighted_punishment_alt_std) for:Machine learning models (e.g., SVM, regression)/Clustering or dimensionality reduction (e.g., PCA, k-means).
- Use both weighted_punishment_std and weighted_punishment_alt_std as features in models and compare their performance using metrics such as R^2, RMSE, or AIC/BIC.

Create delayed cases

In [21]:
```
# Create clean subset with delayed cases
df_delayed = df_filtered[df_filtered['time_delay_days'] > 0].copy()

# Verify distributions in delayed subset
print("\nTarget variable distribution in delayed cases:")
print(df_delayed['weighted_punishment_std'].describe())
```

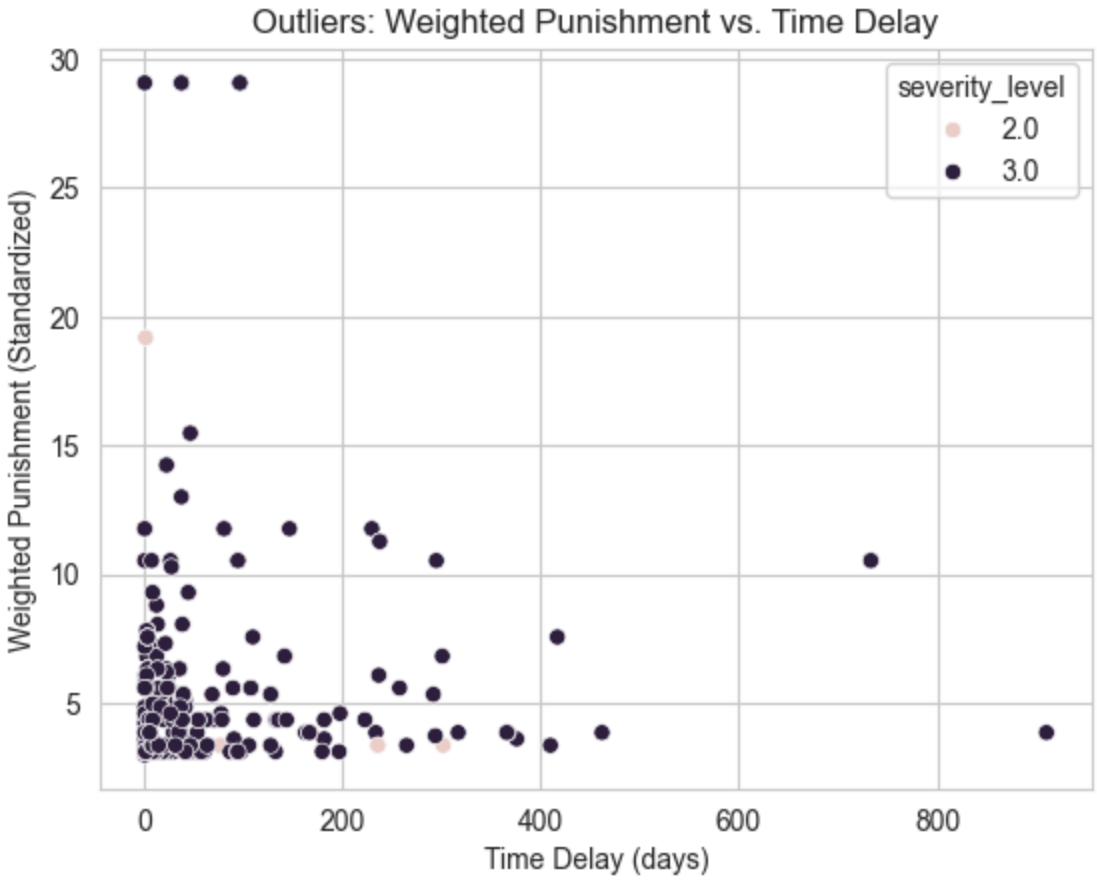
```
Target variable distribution in delayed cases:
count    9290.000000
mean        0.273203
std         1.571977
min        -0.582546
25%        -0.253074
50%        -0.253074
75%         0.405870
max        29.069931
Name: weighted_punishment_std, dtype: float64
```

- The distribution appears right-skewed (mean > median)
- There may be some extreme outliers (max is ~29 standard deviations above mean)
- The 25th and 50th percentiles are identical (-0.253), suggesting a concentration of values

In [22]:
```
outliers = df_delayed[df_delayed['weighted_punishment_std'] > 3]  # Threshol
print(f"Number of outliers: {len(outliers)}")
print(outliers[['weighted_punishment_std', 'time_delay_days', 'severity_leve
```

```
Number of outliers: 283
        weighted_punishment_std  time_delay_days  severity_level
258496                 5.100846               33             3.0
258682                 4.606638               10             3.0
258683                 4.359534               10             3.0
258684                 4.359534               10             3.0
258774                 3.124014                3             3.0
...                         ...              ...             ...
301160                 3.865326              910             3.0
301474                19.185772                2             2.0
301991                 4.359534               39             3.0
302565                 3.371118               32             3.0
303363                 4.606638               27             3.0

[283 rows x 3 columns]
```

In [23]:
```
# outlier detect
sns.scatterplot(data=outliers, x='time_delay_days', y='weighted_punishment_s
plt.title('Outliers: Weighted Punishment vs. Time Delay')
plt.xlabel('Time Delay (days)')
plt.ylabel('Weighted Punishment (Standardized)')
plt.show()
```

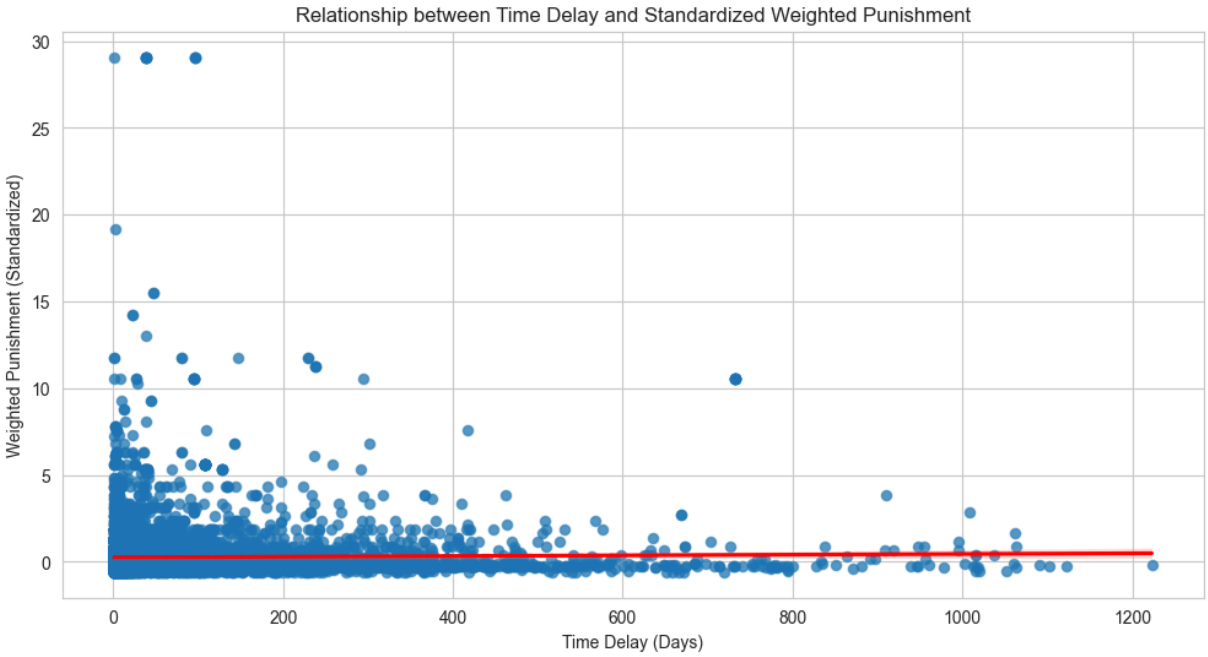


Outlier analysis of punishment severity and case delay:

This figure plots observations with standardized weighted punishment above 3 against time delay (days). Outliers are predominantly associated with higher severity levels, suggesting that extreme punishment outcomes are concentrated among more severe sentencing categories rather than being driven solely by unusually long delays.

EDA

In [24]:
```
# Relationship between time delay and weighted punishment(standardized)
plt.figure(figsize=(12, 6))

sns.scatterplot(data=df_delayed, x='time_delay_days', y='weighted_punishment
sns.regplot(data=df_delayed, x='time_delay_days', y='weighted_punishment_sto

plt.title('Relationship between Time Delay and Standardized Weighted Punishm
plt.xlabel('Time Delay (Days)')  # This explicitly sets the x-label
plt.ylabel('Weighted Punishment (Standardized)')

plt.show()
```

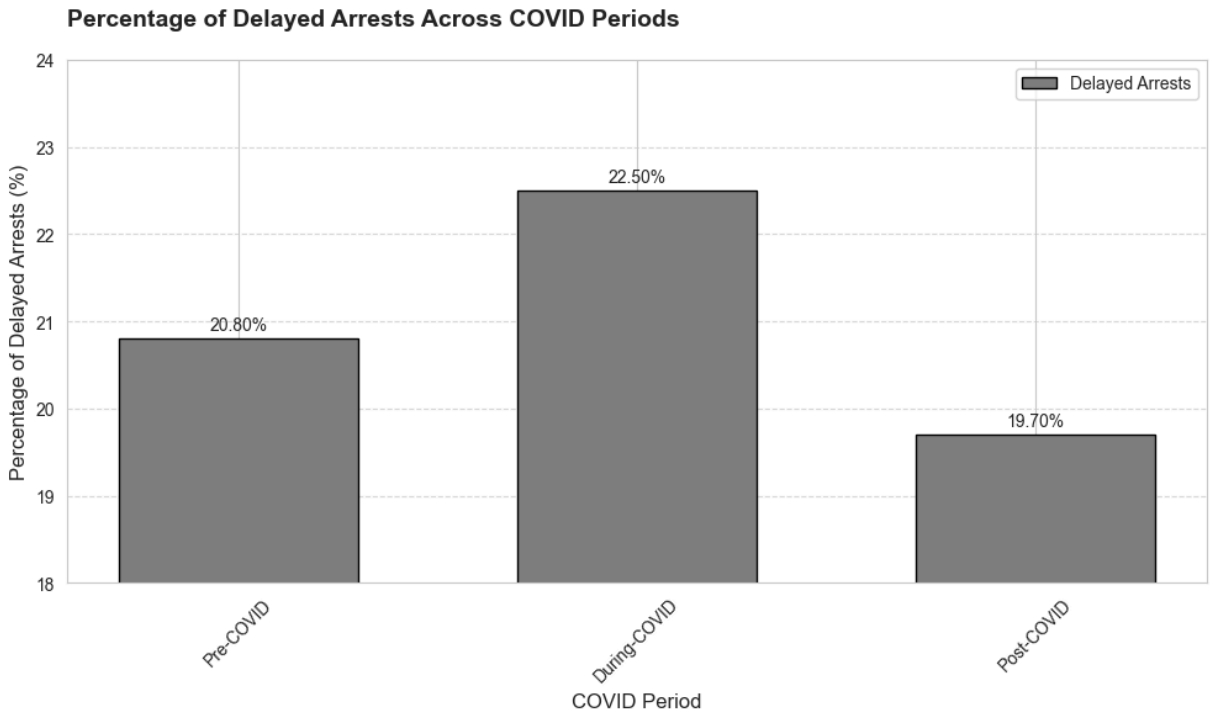


Relationship between case delay and standardized weighted punishment.

Each point represents a case, plotting time delay (days) against standardized weighted punishment. The fitted linear trend (red line) is relatively flat, indicating little systematic association between longer delays and punishment severity for the majority of cases. Extreme punishment values occur primarily at shorter delays, suggesting that unusually severe outcomes are not driven by prolonged case processing time.

In [25]:
```
# Percentage of COVID periods
delayed_period_percentages = {
    'Pre-COVID': 20.8,
    'During-COVID': 22.5,
    'Post-COVID': 19.7
}

labels = list(delayed_period_percentages.keys())
values = list(delayed_period_percentages.values())
x = np.arange(len(labels))
bar_width = 0.6

# Create the plot
plt.figure(figsize=(10, 6))

# Create bars
bars = plt.bar(x, values, width=bar_width, color='grey', edgecolor='black',

# Add value labels
for i, v in enumerate(values):
    plt.text(x[i], v + 0.1, f'{v:.2f}%', ha='center', fontsize=10)

# Customize x-axis and y-axis
plt.xticks(x, labels, fontsize=10, rotation=45)
plt.xlabel('COVID Period', fontsize=12)
```

```
plt.ylabel('Percentage of Delayed Arrests (%)', fontsize=12)

# Set y-axis range with fine grid lines
plt.ylim(18, 24)
plt.gca().yaxis.set_major_locator(plt.MultipleLocator(1))  # More frequent g

plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.title('Percentage of Delayed Arrests Across COVID Periods', loc='left',
plt.legend(loc='upper right', fontsize=10)

plt.tight_layout()
plt.show()
```

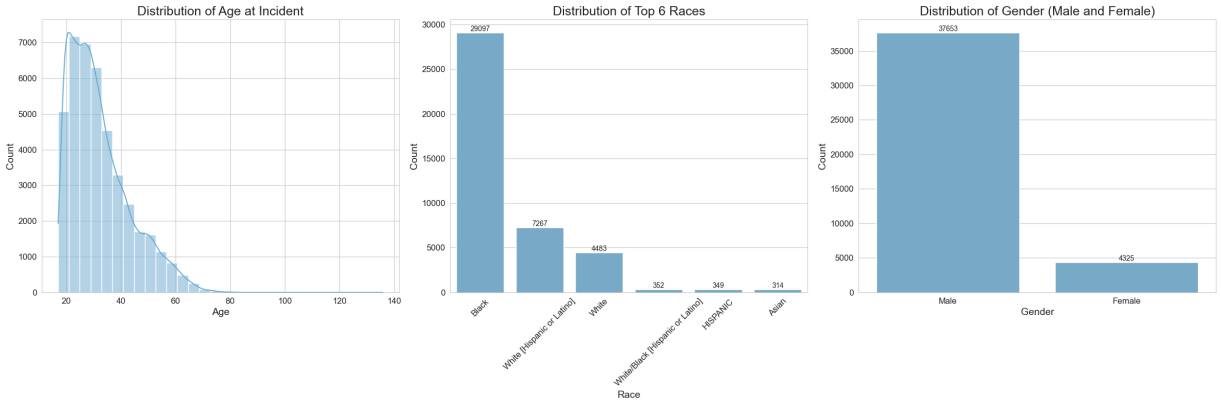


Delayed arrests increase during the COVID period.

The share of arrests classified as delayed rises from 20.8% in the pre-COVID period to 22.5% during COVID, before declining to 19.7% post-COVID. This pattern suggests a temporary disruption to case processing during the pandemic rather than a persistent shift in delay rates.

In [26]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Group all non-binary gender values as "Unknown"
df_clean.loc[:, 'GENDER'] = df_clean['GENDER'].apply(lambda x: x if x in ['F

# Filter gender distribution to show only Male and Female
df_gender_filtered = df_clean[df_clean['GENDER'].isin(['Female', 'Male'])]

# Set figure layout
fig, axes = plt.subplots(1, 3, figsize=(24, 8))  # Adjust height and width f
```

```
# Define font sizes
title_fontsize = 18
label_fontsize = 14
tick_fontsize = 12
value_fontsize = 10

color = '#6baed6'

# 1. Distribution of Age at Incident
sns.histplot(df_clean['AGE_AT_INCIDENT'], bins=30, kde=True, ax=axes[0], col
axes[0].set_title('Distribution of Age at Incident', fontsize=title_fontsize
axes[0].set_xlabel('Age', fontsize=label_fontsize)
axes[0].set_ylabel('Count', fontsize=label_fontsize)
axes[0].tick_params(axis='both', which='major', labelsize=tick_fontsize)

# 2. Distribution of Top 6 Races
top_6_races = df_clean['RACE'].value_counts().head(6).index
df_top_6_races = df_clean[df_clean['RACE'].isin(top_6_races)]
sns.countplot(data=df_top_6_races, x='RACE', ax=axes[1], order=top_6_races,
axes[1].set_title('Distribution of Top 6 Races', fontsize=title_fontsize)
axes[1].set_xlabel('Race', fontsize=label_fontsize)
axes[1].set_ylabel('Count', fontsize=label_fontsize)
axes[1].tick_params(axis='x', rotation=45, labelsize=tick_fontsize)
axes[1].tick_params(axis='y', labelsize=tick_fontsize)

# Add value labels for the Race plot
for p in axes[1].patches:
    axes[1].annotate(f'{int(p.get_height())}',
                     (p.get_x() + p.get_width() / 2., p.get_height()),
                     ha='center', va='bottom', fontsize=value_fontsize)

# 3. Distribution of Gender
sns.countplot(data=df_gender_filtered, x='GENDER', ax=axes[2], order=['Male'
axes[2].set_title('Distribution of Gender (Male and Female)', fontsize=title
axes[2].set_xlabel('Gender', fontsize=label_fontsize)
axes[2].set_ylabel('Count', fontsize=label_fontsize)
axes[2].tick_params(axis='both', which='major', labelsize=tick_fontsize)

# Add value labels for the Gender plot
for p in axes[2].patches:
    axes[2].annotate(f'{int(p.get_height())}',
                     (p.get_x() + p.get_width() / 2., p.get_height()),
                     ha='center', va='bottom', fontsize=value_fontsize)

plt.tight_layout()
plt.show()
```

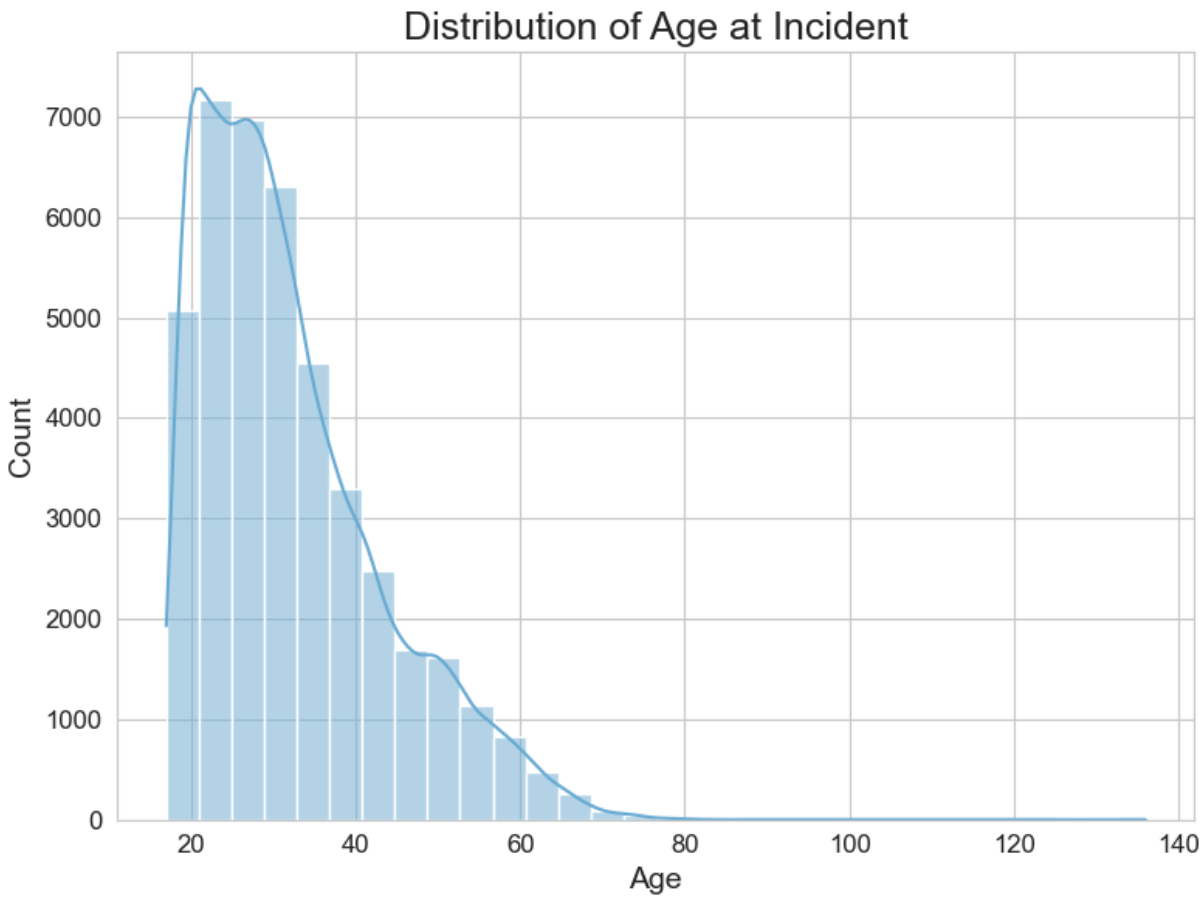


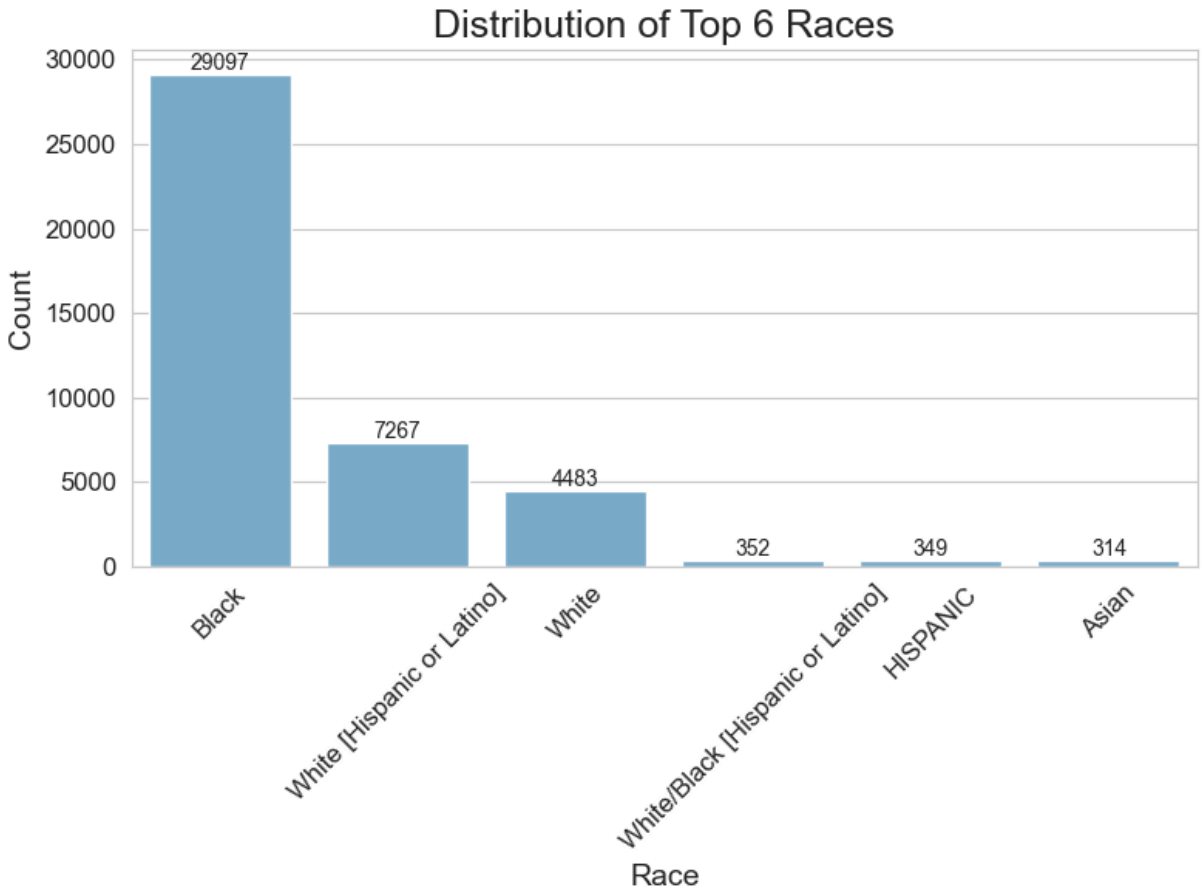
Separate demographic EDA

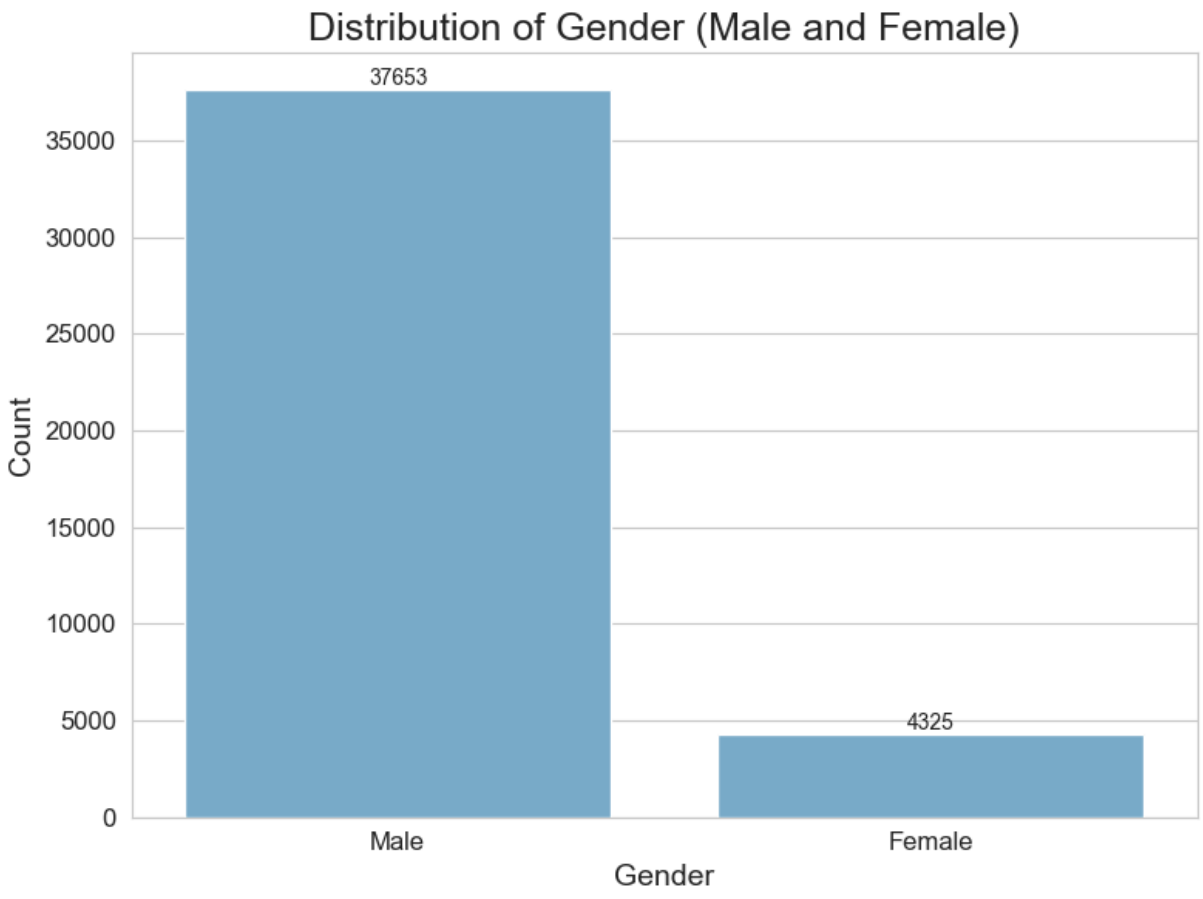
In [27]:

```
import matplotlib.pyplot as plt
import seaborn as sns

# Group all non-binary gender values as "Unknown"
df_clean.loc[:, 'GENDER'] = df_clean['GENDER'].apply(lambda x: x if x in ['F

# Filter gender distribution to show only Male and Female
df_gender_filtered = df_clean[df_clean['GENDER'].isin(['Female', 'Male'])]

# Define common styles
title_fontsize = 18
label_fontsize = 14
tick_fontsize = 12
value_fontsize = 10
color = '#6baed6'

# Distribution of Age at Incident
plt.figure(figsize=(8, 6))
sns.histplot(df_clean['AGE_AT_INCIDENT'], bins=30, kde=True, color=color)
plt.title('Distribution of Age at Incident', fontsize=title_fontsize)
plt.xlabel('Age', fontsize=label_fontsize)
plt.ylabel('Count', fontsize=label_fontsize)
plt.tick_params(axis='both', which='major', labelsize=tick_fontsize)
plt.tight_layout()
plt.show()

# Distribution of Top 6 Races
plt.figure(figsize=(8, 6))
top_6_races = df_clean['RACE'].value_counts().head(6).index
df_top_6_races = df_clean[df_clean['RACE'].isin(top_6_races)]
sns.countplot(data=df_top_6_races, x='RACE', order=top_6_races, color=color)
plt.title('Distribution of Top 6 Races', fontsize=title_fontsize)
plt.xlabel('Race', fontsize=label_fontsize)
plt.ylabel('Count', fontsize=label_fontsize)
plt.tick_params(axis='x', rotation=45, labelsize=tick_fontsize)
plt.tick_params(axis='y', labelsize=tick_fontsize)

# Add value labels
for p in plt.gca().patches:
    plt.text(p.get_x() + p.get_width() / 2., p.get_height(),
```

```
            f'{int(p.get_height())}', ha='center', va='bottom', fontsize=va

plt.tight_layout()
plt.show()

# Distribution of Gender
plt.figure(figsize=(8, 6))
sns.countplot(data=df_gender_filtered, x='GENDER', order=['Male', 'Female'],
plt.title('Distribution of Gender (Male and Female)', fontsize=title_fontsiz
plt.xlabel('Gender', fontsize=label_fontsize)
plt.ylabel('Count', fontsize=label_fontsize)
plt.tick_params(axis='both', which='major', labelsize=tick_fontsize)

# Add value labels
for p in plt.gca().patches:
    plt.text(p.get_x() + p.get_width() / 2., p.get_height(),
            f'{int(p.get_height())}', ha='center', va='bottom', fontsize=va

plt.tight_layout()
plt.show()
```







Demographic characteristics of the sample.

The age distribution is right-skewed, with most incidents involving individuals in early adulthood and steadily declining frequencies at older ages. The racial composition is highly uneven, with Black individuals comprising the largest share of cases, followed by White Hispanic/Latino and White individuals. The sample is also heavily male-dominated, with males accounting for the vast majority of observations.

Regression Model: Repetitive study

This regression replicates the original specification to assess the robustness of the main findings under the same modeling framework.

Missing values for std

In [28]:
```
# Check for missing values
print(df_delayed[['time_delay_days', 'weighted_punishment_std']].isna().sum(

# Check for infinite values
print((df_delayed[['time_delay_days', 'weighted_punishment_std']] == np.inf)
print((df_delayed[['time_delay_days', 'weighted_punishment_std']] == -np.inf

df_delayed = df_delayed.dropna(subset=['weighted_punishment_std'])
print(f"Remaining observations: {len(df_delayed)}")
```

```
time_delay_days             0
weighted_punishment_std    96
dtype: int64
time_delay_days            0
weighted_punishment_std    0
dtype: int64
time_delay_days            0
weighted_punishment_std    0
dtype: int64
Remaining observations: 9290
```

Interpretation

The results suggest that time_delay_days alone is not a strong predictor of weighted_punishment_std. However, this does not necessarily mean that delay has no effect—it could indicate that:

- The relationship is non-linear.
- Other variables (e.g., severity_level, covid_period) interact with time_delay_days or confound the relationship.
- Outliers or data distribution issues (e.g., skewness) are obscuring the relationship.

In [29]:
```
print("Number of unique categories:")
print(f"RACE: {df_delayed['RACE'].nunique()}")
print(f"UPDATED_OFFENSE_CATEGORY: {df_delayed['UPDATED_OFFENSE_CATEGORY'].nu
print(f"SENTENCE_JUDGE: {df_delayed['SENTENCE_JUDGE'].nunique()}")
```

```
# Display detailed breakdown
print("\nRACE categories:")
print(df_delayed['RACE'].value_counts())

print("\nTop 10 offense categories:")
print(df_delayed['UPDATED_OFFENSE_CATEGORY'].value_counts().head(10))
```

```
Number of unique categories:
RACE: 10
UPDATED_OFFENSE_CATEGORY: 64
SENTENCE_JUDGE: 134

RACE categories:
RACE
Black                                5606
White                                1674
White [Hispanic or Latino]           1660
Asian                                 116
White/Black [Hispanic or Latino]       83
HISPANIC                               62
Unknown                                33
American Indian                         7
Middle Eastern/North African            2
Biracial                                1
Name: count, dtype: int64

Top 10 offense categories:
UPDATED_OFFENSE_CATEGORY
Burglary                              1116
Narcotics                              693
UUW - Unlawful Use of Weapon           668
Theft                                  482
Sex Crimes                             446
Aggravated Battery                     409
Armed Robbery                          382
Attempt Homicide                       330
Possession of Stolen Motor Vehicle     321
Robbery                                316
Name: count, dtype: int64
```

In [30]:
```
# Get all unique crime categories
unique_crimes = df_delayed['UPDATED_OFFENSE_CATEGORY'].unique()

# Display the unique values
print("Unique Crime Categories:")
for crime in unique_crimes:
    print(crime)

# Count the total number of unique categories
print(f"\nTotal unique crime categories: {len(unique_crimes)}")
```

Unique Crime Categories:
Other Offense
Aggravated Battery Police Officer
Aggravated Robbery
Burglary
Attempt Armed Robbery
Fraudulent ID
Domestic Battery
Reckless Discharge of Firearm
Aggravated Battery
Criminal Damage to Property
Escape – Failure to Return
Attempt Homicide
Identity Theft
Theft
Armed Robbery
Home Invasion
Robbery
Aggravated Fleeing and Eluding
UUW – Unlawful Use of Weapon
Residential Burglary
Retail Theft
Homicide
Sex Crimes
Aggravated DUI
Narcotics
Vehicular Hijacking
Possession of Stolen Motor Vehicle
Obstructing Justice
Failure to Register as a Sex Offender
Violation Order Of Protection
Forgery
Child Pornography
Aggravated Battery With A Firearm
Credit Card Cases
Kidnapping
Intimidation
Aggravated Assault Police Officer
Child Abduction
Aggravated Discharge Firearm
Stalking
Driving With Suspended Or Revoked License
Criminal Trespass To Residence
Vehicular Invasion
Reckless Homicide
Gun – Non UUW
Arson
Impersonating Police Officer
Hate Crimes
Major Accidents
Attempt Arson
Possession of Contraband in Penal Institution
Official Misconduct
Fraud
Prostitution
Possession of Explosives

```
Human Trafficking
Deceptive Practice
Unlawful Restraint
Attempt Vehicular Hijacking
Bribery
Bomb Threat
Communicating With Witness
Disarming Police Officer
Tampering

Total unique crime categories: 64
```

In [31]:
```python
df_delayed = df_delayed.copy()

def simplify_race(race):
    if race == 'White':
        return 'White'
    elif race in ['White [Hispanic or Latino]', 'White/Black [Hispanic or La
        return 'Hispanic'
    elif race == 'Black':
        return 'Black'
    elif race == 'Asian':
        return 'Asian'
    else:
        return 'Other'

df_delayed.loc[:, 'RACE_simplified'] = df_delayed['RACE'].apply(simplify_rac

print("\nSimplified Race Categories:")
print(df_delayed['RACE_simplified'].value_counts(normalize=True))
```

```
Simplified Race Categories:
RACE_simplified
Black       0.603445
Hispanic    0.194295
White       0.180194
Asian       0.012487
Other       0.009580
Name: proportion, dtype: float64
```

In [32]:
```python
def prepare_data(df_delayed):
    # Group all non-binary or unknown gender values as "Unknown"
    df_delayed['GENDER'] = df_delayed['GENDER'].apply(lambda x: x if x in ['

    # Convert categorical columns to 'category' dtype for memory efficiency
    df_delayed['UPDATED_OFFENSE_CATEGORY'] = df_delayed['UPDATED_OFFENSE_CAT
    df_delayed['SENTENCE_JUDGE'] = df_delayed['SENTENCE_JUDGE'].astype('cate

    # Create dummies for GENDER, but only for Male and Female
    df_delayed = pd.get_dummies(df_delayed, columns=['GENDER'], drop_first=T

    df_delayed['LENGTH_OF_CASE_in_Days'] = pd.to_numeric(df_delayed['LENGTH_

    # Scale numerical features
    scaler = StandardScaler()
    numerical_features = ['CHARGE_COUNT', 'AGE_AT_INCIDENT', 'LENGTH_OF_CASE
```

```
    df_delayed[numerical_features] = scaler.fit_transform(df_delayed[numeric

    return df_delayed
```

In [35]:

```
# Modified regression formula keeping original crime types and judge categor
df_delayed['log_time_delay'] = np.log1p(df_delayed['time_delay_days'])

df_delayed['LENGTH_OF_CASE_in_Days'] = pd.to_numeric(df_delayed['LENGTH_OF_C

formula = ('weighted_punishment_std ~ log_time_delay + C(UPDATED_OFFENSE_CAT
           'CHARGE_COUNT * LENGTH_OF_CASE_in_Days + C(RACE_simplified) + AGE
           'C(SENTENCE_JUDGE)  + covid_period_num')

# Fit the model
model = sm.OLS.from_formula(formula, data=df_delayed)
results = model.fit()

# Print regression results
print("\nRegression Results:")
print(results.summary())

# Save and print key coefficients and p-values
coef_log_time_delay = results.params['log_time_delay']
pval_log_time_delay = results.pvalues['log_time_delay']

# Summarized regression results
print("\nKey Metrics and Results:")
print(f"R-squared: {results.rsquared:.4f}")
print(f"Adjusted R-squared: {results.rsquared_adj:.4f}")
print(f"AIC: {results.aic:.2f}")
print(f"BIC: {results.bic:.2f}")

# Log(Time Delay) Effect
if 'log_time_delay' in results.params:
    coef_log_time_delay = results.params['log_time_delay']
    pval_log_time_delay = results.pvalues['log_time_delay']
    print(f"\nLog(Time Delay) Effect: {coef_log_time_delay:.4f} (p = {pval_l

# Optional: Save important coefficients and p-values for a few selected vari
selected_vars = ['log_time_delay', 'CHARGE_COUNT', 'AGE_AT_INCIDENT', 'covic
                 'C(RACE_simplified)[T.Black]', 'C(RACE_simplified)[T.Hispar
                 'C(RACE_simplified)[T.Other]', 'C(GENDER)[T.Male]']

print("\nSelected Coefficients and P-values:")
for var in selected_vars:
    if var in results.params:
        print(f"{var}: Coef = {results.params[var]:.4f}, p-value = {results.

# Add a note about omitted coefficients
print("\nNote: Detailed coefficients for categorical variables and interacti
```

Regression Results:

```
                            OLS Regression Results
==============================================================================
=======
Dep. Variable:     weighted_punishment_std   R-squared:
0.360
Model:                                 OLS   Adj. R-squared:
0.345
Method:                      Least Squares   F-statistic:
24.44
Date:                     Thu, 05 Feb 2026   Prob (F-statistic):
0.00
Time:                             22:46:30   Log-Likelihood:
-13018.
No. Observations:                     8908   AIC:                          2.
644e+04
Df Residuals:                         8707   BIC:                          2.
786e+04
Df Model:                              200
Covariance Type:                 nonrobust
==============================================================================
==========================================================================

coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
------------------------------------------------------------------------
Intercept
-1.2188      1.093     -1.115      0.265      -3.362       0.924
C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated Battery]
0.0645      0.195      0.330      0.741      -0.318       0.447
C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated Battery Police Officer]
0.0794      0.199      0.400      0.689      -0.310       0.469
C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated Battery With A Firearm]
0.6389      0.258      2.475      0.013       0.133       1.145
C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated DUI]
0.2676      0.205      1.305      0.192      -0.134       0.669
C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated Discharge Firearm]
0.2150      0.212      1.015      0.310      -0.200       0.630
C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated Fleeing and Eluding]
0.0111      0.200      0.056      0.956      -0.380       0.403
C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated Robbery]
0.8501      0.211      4.030      0.000       0.437       1.264
C(UPDATED_OFFENSE_CATEGORY)[T.Armed Robbery]
1.2565      0.196      6.416      0.000       0.873       1.640
C(UPDATED_OFFENSE_CATEGORY)[T.Arson]
0.3301      0.218      1.517      0.129      -0.096       0.757
C(UPDATED_OFFENSE_CATEGORY)[T.Attempt Armed Robbery]
0.8187      0.312      2.621      0.009       0.206       1.431
C(UPDATED_OFFENSE_CATEGORY)[T.Attempt Arson]
0.2792      0.341      0.820      0.412      -0.389       0.947
C(UPDATED_OFFENSE_CATEGORY)[T.Attempt Homicide]
1.4091      0.199      7.077      0.000       1.019       1.799
C(UPDATED_OFFENSE_CATEGORY)[T.Attempt Vehicular Hijacking]
1.5167      0.569      2.668      0.008       0.402       2.631
C(UPDATED_OFFENSE_CATEGORY)[T.Bomb Threat]
0.0173      0.674      0.026      0.980      -1.304       1.338
```

```
C(UPDATED_OFFENSE_CATEGORY)[T.Bribery]
0.0804      1.075      0.075      0.940      -2.026       2.187
C(UPDATED_OFFENSE_CATEGORY)[T.Burglary]
0.3198      0.191      1.677      0.094      -0.054       0.694
C(UPDATED_OFFENSE_CATEGORY)[T.Child Abduction]
-0.1078      0.473     -0.228      0.820      -1.036       0.820
C(UPDATED_OFFENSE_CATEGORY)[T.Child Pornography]
0.1914      0.228      0.838      0.402      -0.256       0.639
C(UPDATED_OFFENSE_CATEGORY)[T.Communicating With Witness]
-0.0501      0.640     -0.078      0.938      -1.305       1.205
C(UPDATED_OFFENSE_CATEGORY)[T.Credit Card Cases]
0.1187      0.255      0.465      0.642      -0.381       0.618
C(UPDATED_OFFENSE_CATEGORY)[T.Criminal Damage to Property]
-0.0468      0.201     -0.233      0.816      -0.440       0.347
C(UPDATED_OFFENSE_CATEGORY)[T.Criminal Trespass To Residence]
0.0200      0.385      0.052      0.958      -0.734       0.775
C(UPDATED_OFFENSE_CATEGORY)[T.Deceptive Practice]
0.0422      0.778      0.054      0.957      -1.482       1.567
C(UPDATED_OFFENSE_CATEGORY)[T.Disarming Police Officer]
0.2310      0.773      0.299      0.765      -1.284       1.746
C(UPDATED_OFFENSE_CATEGORY)[T.Domestic Battery]
0.1729      0.202      0.855      0.392      -0.223       0.569
C(UPDATED_OFFENSE_CATEGORY)[T.Driving With Suspended Or Revoked License]
0.0250      0.236      0.106      0.916      -0.437       0.487
C(UPDATED_OFFENSE_CATEGORY)[T.Escape - Failure to Return]
-0.0475      0.202     -0.236      0.814      -0.443       0.347
C(UPDATED_OFFENSE_CATEGORY)[T.Failure to Register as a Sex Offender]
-0.0707      0.243     -0.291      0.771      -0.547       0.405
C(UPDATED_OFFENSE_CATEGORY)[T.Forgery]
0.1069      0.217      0.493      0.622      -0.318       0.532
C(UPDATED_OFFENSE_CATEGORY)[T.Fraud]
-0.0056      0.308     -0.018      0.986      -0.610       0.599
C(UPDATED_OFFENSE_CATEGORY)[T.Fraudulent ID]
-0.1237      0.419     -0.295      0.768      -0.945       0.698
C(UPDATED_OFFENSE_CATEGORY)[T.Gun - Non UUW]
-0.2011      0.562     -0.358      0.720      -1.302       0.900
C(UPDATED_OFFENSE_CATEGORY)[T.Hate Crimes]
-0.0322      0.314     -0.102      0.919      -0.648       0.584
C(UPDATED_OFFENSE_CATEGORY)[T.Home Invasion]
0.6234      0.216      2.887      0.004       0.200       1.047
C(UPDATED_OFFENSE_CATEGORY)[T.Homicide]
5.0753      0.215     23.555      0.000       4.653       5.498
C(UPDATED_OFFENSE_CATEGORY)[T.Human Trafficking]
1.5777      0.563      2.802      0.005       0.474       2.682
C(UPDATED_OFFENSE_CATEGORY)[T.Identity Theft]
0.1309      0.199      0.658      0.511      -0.259       0.521
C(UPDATED_OFFENSE_CATEGORY)[T.Impersonating Police Officer]
0.0492      0.401      0.123      0.902      -0.736       0.835
C(UPDATED_OFFENSE_CATEGORY)[T.Intimidation]
0.4707      0.299      1.577      0.115      -0.115       1.056
C(UPDATED_OFFENSE_CATEGORY)[T.Kidnapping]
0.4186      0.276      1.516      0.129      -0.123       0.960
C(UPDATED_OFFENSE_CATEGORY)[T.Major Accidents]
0.5823      0.275      2.114      0.035       0.042       1.122
C(UPDATED_OFFENSE_CATEGORY)[T.Narcotics]
-0.0370      0.193     -0.192      0.848      -0.415       0.341
```

C(UPDATED_OFFENSE_CATEGORY)[T.Obstructing Justice]
-0.1097 0.564 -0.195 0.846 -1.215 0.995
C(UPDATED_OFFENSE_CATEGORY)[T.Official Misconduct]
-0.3231 0.772 -0.418 0.676 -1.837 1.191
C(UPDATED_OFFENSE_CATEGORY)[T.Other Offense]
-0.0215 0.200 -0.108 0.914 -0.413 0.370
C(UPDATED_OFFENSE_CATEGORY)[T.Possession of Contraband in Penal Institution]
0.1116 0.314 0.355 0.722 -0.504 0.727
C(UPDATED_OFFENSE_CATEGORY)[T.Possession of Explosives]
0.0902 1.076 0.084 0.933 -2.019 2.200
C(UPDATED_OFFENSE_CATEGORY)[T.Possession of Stolen Motor Vehicle]
0.1914 0.197 0.971 0.331 -0.195 0.578
C(UPDATED_OFFENSE_CATEGORY)[T.Prostitution]
-0.2522 0.563 -0.448 0.654 -1.356 0.852
C(UPDATED_OFFENSE_CATEGORY)[T.Reckless Discharge of Firearm]
-0.0217 0.228 -0.095 0.924 -0.468 0.424
C(UPDATED_OFFENSE_CATEGORY)[T.Reckless Homicide]
0.3847 0.340 1.130 0.259 -0.283 1.052
C(UPDATED_OFFENSE_CATEGORY)[T.Residential Burglary]
0.3676 0.202 1.815 0.070 -0.029 0.764
C(UPDATED_OFFENSE_CATEGORY)[T.Retail Theft]
0.1141 0.198 0.575 0.565 -0.275 0.503
C(UPDATED_OFFENSE_CATEGORY)[T.Robbery]
0.3116 0.197 1.579 0.114 -0.075 0.698
C(UPDATED_OFFENSE_CATEGORY)[T.Sex Crimes]
0.7859 0.196 4.015 0.000 0.402 1.170
C(UPDATED_OFFENSE_CATEGORY)[T.Stalking]
0.0551 0.228 0.242 0.809 -0.392 0.502
C(UPDATED_OFFENSE_CATEGORY)[T.Tampering]
-0.0108 0.775 -0.014 0.989 -1.530 1.508
C(UPDATED_OFFENSE_CATEGORY)[T.Theft]
0.0930 0.195 0.478 0.633 -0.288 0.474
C(UPDATED_OFFENSE_CATEGORY)[T.UUW - Unlawful Use of Weapon]
0.2945 0.192 1.531 0.126 -0.083 0.672
C(UPDATED_OFFENSE_CATEGORY)[T.Unlawful Restraint]
0.2818 0.308 0.915 0.360 -0.322 0.885
C(UPDATED_OFFENSE_CATEGORY)[T.Vehicular Hijacking]
1.2854 0.206 6.247 0.000 0.882 1.689
C(UPDATED_OFFENSE_CATEGORY)[T.Vehicular Invasion]
0.1399 0.283 0.494 0.621 -0.415 0.695
C(UPDATED_OFFENSE_CATEGORY)[T.Violation Order Of Protection]
0.0134 0.300 0.045 0.964 -0.574 0.601
C(RACE_simplified)[T.Black]
0.1885 0.103 1.834 0.067 -0.013 0.390
C(RACE_simplified)[T.Hispanic]
0.1288 0.104 1.235 0.217 -0.076 0.333
C(RACE_simplified)[T.Other]
-0.1728 0.162 -1.065 0.287 -0.491 0.145
C(RACE_simplified)[T.White]
0.1856 0.104 1.783 0.075 -0.018 0.390
C(GENDER)[T.Male]
0.2489 0.034 7.244 0.000 0.182 0.316
C(SENTENCE_JUDGE)[T.Adrienne Davis]
0.1401 1.078 0.130 0.897 -1.973 2.253
C(SENTENCE_JUDGE)[T.Aleksandra Gillespie]
0.1934 1.073 0.180 0.857 -1.911 2.297

C(SENTENCE_JUDGE)[T.Alfredo Maldonado]
0.3903   1.074   0.363   0.716   -1.715   2.495
C(SENTENCE_JUDGE)[T.Andreana A Turano]
4.272e-14   3.61e-14   1.184   0.236   -2.8e-14   1.13e-13
C(SENTENCE_JUDGE)[T.Angela Munari Petrone]
0.1893   1.074   0.176   0.860   -1.916   2.294
C(SENTENCE_JUDGE)[T.Anjana Hansen]
0.0820   1.071   0.077   0.939   -2.018   2.182
C(SENTENCE_JUDGE)[T.Anthony John Calabrese]
2.968e-14   2.57e-14   1.154   0.249   -2.08e-14   8.01e-14
C(SENTENCE_JUDGE)[T.Arthur F Hill]
0.3172   1.117   0.284   0.776   -1.872   2.507
C(SENTENCE_JUDGE)[T.Arthur Wesley Willis]
0.4584   1.074   0.427   0.669   -1.646   2.563
C(SENTENCE_JUDGE)[T.Barbara Lynette Dawkins]
-0.4435   1.170   -0.379   0.705   -2.737   1.850
C(SENTENCE_JUDGE)[T.Brian K Flaherty]
0.0774   1.085   0.071   0.943   -2.050   2.204
C(SENTENCE_JUDGE)[T.Bridget Jane Hughes]
0.6040   1.504   0.402   0.688   -2.344   3.552
C(SENTENCE_JUDGE)[T.Byrne, Thomas]
1.0376   1.073   0.967   0.334   -1.066   3.141
C(SENTENCE_JUDGE)[T.Carl Anthony Walker]
-0.3663   1.504   -0.244   0.808   -3.314   2.582
C(SENTENCE_JUDGE)[T.Carl B Boyd]
-0.0785   1.074   -0.073   0.942   -2.183   2.026
C(SENTENCE_JUDGE)[T.Carmen Aguilar]
-0.1807   1.092   -0.165   0.869   -2.322   1.960
C(SENTENCE_JUDGE)[T.Carol M Howard]
0.4022   1.074   0.374   0.708   -1.704   2.508
C(SENTENCE_JUDGE)[T.Catherine Marie Haberkorn]
0.2032   1.076   0.189   0.850   -1.906   2.313
C(SENTENCE_JUDGE)[T.Charles P Burns]
0.3663   1.074   0.341   0.733   -1.739   2.472
C(SENTENCE_JUDGE)[T.Cheyrl D Ingram]
1.0386   1.505   0.690   0.490   -1.912   3.989
C(SENTENCE_JUDGE)[T.Colleen Ann Hyland]
0.1372   1.121   0.122   0.903   -2.060   2.334
C(SENTENCE_JUDGE)[T.Daniel Gallagher]
-0.3241   1.305   -0.248   0.804   -2.882   2.234
C(SENTENCE_JUDGE)[T.Daniel E Maloney]
-0.1172   1.142   -0.103   0.918   -2.356   2.122
C(SENTENCE_JUDGE)[T.David P Sterba]
1.2391   1.504   0.824   0.410   -1.708   4.186
C(SENTENCE_JUDGE)[T.Demetrios G Kottaras]
-0.2264   1.503   -0.151   0.880   -3.173   2.721
C(SENTENCE_JUDGE)[T.Dennis J Porter]
0.4253   1.505   0.283   0.777   -2.524   3.375
C(SENTENCE_JUDGE)[T.Domenica A Stephenson]
0.6310   1.074   0.588   0.557   -1.473   2.735
C(SENTENCE_JUDGE)[T.ELLEN MANDELTORT]
2.803e-15   4e-15   0.700   0.484   -5.04e-15   1.06e-14
C(SENTENCE_JUDGE)[T.Elizabeth Ciaccia-Lezza]
-0.1748   1.113   -0.157   0.875   -2.356   2.006
C(SENTENCE_JUDGE)[T.Eric M Sauceda]
0.2706   1.232   0.220   0.826   -2.144   2.685

C(SENTENCE_JUDGE)[T.Erica L Reddick]
0.7948 1.505 0.528 0.597 -2.154 3.744
C(SENTENCE_JUDGE)[T.Eulalia V. De La Rosa]
0.0961 1.074 0.089 0.929 -2.010 2.202
C(SENTENCE_JUDGE)[T.Frank John Andreou]
0.1098 1.307 0.084 0.933 -2.452 2.672
C(SENTENCE_JUDGE)[T.Geary W Kull]
-0.0058 1.071 -0.005 0.996 -2.105 2.093
C(SENTENCE_JUDGE)[T.Geraldine A D'Souza]
-0.0295 1.073 -0.028 0.978 -2.133 2.074
C(SENTENCE_JUDGE)[T.Gregory Paul Vazquez]
0.1463 1.074 0.136 0.892 -1.959 2.252
C(SENTENCE_JUDGE)[T.James Novy]
0.4694 1.085 0.433 0.665 -1.657 2.595
C(SENTENCE_JUDGE)[T.James B Linn]
0.4174 1.075 0.388 0.698 -1.690 2.524
C(SENTENCE_JUDGE)[T.James Michael Obbish]
0.4833 1.075 0.450 0.653 -1.624 2.591
C(SENTENCE_JUDGE)[T.Janet Adams Brosnahan]
-0.2838 1.134 -0.250 0.802 -2.506 1.939
C(SENTENCE_JUDGE)[T.Jeanine Wrenn]
-0.1761 1.506 -0.117 0.907 -3.128 2.776
C(SENTENCE_JUDGE)[T.Jennifer Duncan-Brice]
-6.225e-16 1.61e-15 -0.387 0.699 -3.77e-15 2.53e-15
C(SENTENCE_JUDGE)[T.Jennifer Frances Coleman]
0.1851 1.089 0.170 0.865 -1.950 2.320
C(SENTENCE_JUDGE)[T.Jill Marisie]
0.0313 1.114 0.028 0.978 -2.153 2.215
C(SENTENCE_JUDGE)[T.Jim Ryan]
-0.0782 1.194 -0.066 0.948 -2.418 2.262
C(SENTENCE_JUDGE)[T.Joan Margaret O'Brien]
0.1414 1.194 0.118 0.906 -2.198 2.481
C(SENTENCE_JUDGE)[T.Joanne Rosado]
0.2284 1.077 0.212 0.832 -1.882 2.339
C(SENTENCE_JUDGE)[T.John F Lyke]
0.0222 1.079 0.021 0.984 -2.092 2.137
C(SENTENCE_JUDGE)[T.John Wellington Wilson]
-0.0488 1.074 -0.045 0.964 -2.155 2.057
C(SENTENCE_JUDGE)[T.Joseph Cataldo]
0.2097 1.070 0.196 0.845 -1.888 2.308
C(SENTENCE_JUDGE)[T.Joseph Michael Claps]
0.3390 1.078 0.314 0.753 -1.775 2.453
C(SENTENCE_JUDGE)[T.Kathaleen Lanahan]
-0.6162 1.232 -0.500 0.617 -3.030 1.798
C(SENTENCE_JUDGE)[T.Kelly Marie McCarthy]
0.1393 1.504 0.093 0.926 -2.809 3.087
C(SENTENCE_JUDGE)[T.Kenneth J Wadas]
0.1440 1.075 0.134 0.893 -1.963 2.251
C(SENTENCE_JUDGE)[T.Kenneth L Fletcher]
0.3382 1.504 0.225 0.822 -2.609 3.286
C(SENTENCE_JUDGE)[T.Kenworthy, Diana]
0.2141 1.074 0.199 0.842 -1.891 2.320
C(SENTENCE_JUDGE)[T.Kerry M Kennedy]
0.1925 1.127 0.171 0.864 -2.016 2.401
C(SENTENCE_JUDGE)[T.Kevin P Cunningham]
-0.1987 1.106 -0.180 0.858 -2.368 1.970

C(SENTENCE_JUDGE)[T.Kristyna C Ryan]
-0.3172 1.194 -0.266 0.790 -2.657 2.023
C(SENTENCE_JUDGE)[T.Lakshmi Elkhanialy Jha]
0.2283 1.142 0.200 0.842 -2.011 2.468
C(SENTENCE_JUDGE)[T.Laura Ayala-Gonzalez]
-0.1142 1.080 -0.106 0.916 -2.230 2.002
C(SENTENCE_JUDGE)[T.Laura Bertucci Smith]
0.2838 1.522 0.186 0.852 -2.700 3.267
C(SENTENCE_JUDGE)[T.Laura M Sullivan]
3.081e-16 1.57e-15 0.196 0.845 -2.78e-15 3.39e-15
C(SENTENCE_JUDGE)[T.Lauren Gottainer Edidin]
0.0348 1.075 0.032 0.974 -2.073 2.143
C(SENTENCE_JUDGE)[T.Lawrence Edward Flood]
0.0893 1.077 0.083 0.934 -2.022 2.201
C(SENTENCE_JUDGE)[T.Lee Preston]
0.3177 1.506 0.211 0.833 -2.635 3.270
C(SENTENCE_JUDGE)[T.Leroy K Martin]
0.3375 1.504 0.224 0.822 -2.611 3.286
C(SENTENCE_JUDGE)[T.Lindsay Huge]
-8.17e-16 1.28e-15 -0.639 0.523 -3.32e-15 1.69e-15
C(SENTENCE_JUDGE)[T.Lisette Catherine Mojica]
0.4610 1.507 0.306 0.760 -2.493 3.415
C(SENTENCE_JUDGE)[T.Lorraine Mary Murphy]
0.1644 1.083 0.152 0.879 -1.958 2.286
C(SENTENCE_JUDGE)[T.Marc W Martin]
0.1262 1.071 0.118 0.906 -1.974 2.226
C(SENTENCE_JUDGE)[T.Marcia Maras]
0.1318 1.505 0.088 0.930 -2.817 3.081
C(SENTENCE_JUDGE)[T.Margaret Ogarek]
0.0996 1.072 0.093 0.926 -2.001 2.200
C(SENTENCE_JUDGE)[T.Maria Kuriakos Ciesil]
0.1813 1.075 0.169 0.866 -1.925 2.288
C(SENTENCE_JUDGE)[T.Mariano Ricardo Reyna]
0.4961 1.094 0.454 0.650 -1.648 2.640
C(SENTENCE_JUDGE)[T.Maritza Martinez]
-0.4389 1.507 -0.291 0.771 -3.393 2.515
C(SENTENCE_JUDGE)[T.Martin Paul Moltz]
-3.528e-16 9.43e-16 -0.374 0.708 -2.2e-15 1.49e-15
C(SENTENCE_JUDGE)[T.Mary Anna Planey]
-0.0734 1.305 -0.056 0.955 -2.631 2.485
C(SENTENCE_JUDGE)[T.Mary C Marubio]
-0.0511 1.306 -0.039 0.969 -2.612 2.509
C(SENTENCE_JUDGE)[T.Mary C Roberts]
0.2612 1.518 0.172 0.863 -2.714 3.236
C(SENTENCE_JUDGE)[T.Mary Margaret Brosnahan]
0.1828 1.075 0.170 0.865 -1.925 2.291
C(SENTENCE_JUDGE)[T.Matthew Carmody]
0.5346 1.507 0.355 0.723 -2.420 3.489
C(SENTENCE_JUDGE)[T.Megan Goldish]
0.0157 1.505 0.010 0.992 -2.935 2.966
C(SENTENCE_JUDGE)[T.Michael Clancy]
0.3502 1.073 0.326 0.744 -1.754 2.454
C(SENTENCE_JUDGE)[T.Michael Kane]
0.5181 1.090 0.475 0.635 -1.619 2.655
C(SENTENCE_JUDGE)[T.Michael B McHale]
0.1242 1.077 0.115 0.908 -1.987 2.235

C(SENTENCE_JUDGE)[T.Michael J Hood]
0.2354 1.072 0.220 0.826 -1.867 2.337
C(SENTENCE_JUDGE)[T.Michael Nando Pattarozzi]
-0.0651 1.504 -0.043 0.965 -3.013 2.883
C(SENTENCE_JUDGE)[T.Michele Ann Gemskie]
-0.0676 1.100 -0.061 0.951 -2.225 2.090
C(SENTENCE_JUDGE)[T.Michele M Pitman]
0.1918 1.074 0.179 0.858 -1.913 2.296
C(SENTENCE_JUDGE)[T.Natosha Cuyler Toller]
0.2104 1.092 0.193 0.847 -1.930 2.351
C(SENTENCE_JUDGE)[T.Neera Walsh]
0.3552 1.074 0.331 0.741 -1.749 2.460
C(SENTENCE_JUDGE)[T.Nicholas Kantas]
-0.0242 1.090 -0.022 0.982 -2.161 2.113
C(SENTENCE_JUDGE)[T.Pamela A Leeming]
0.1567 1.142 0.137 0.891 -2.082 2.396
C(SENTENCE_JUDGE)[T.Pamela J Stratigakis]
0.1330 1.077 0.124 0.902 -1.978 2.244
C(SENTENCE_JUDGE)[T.Patricia M Martin]
1.3177 1.306 1.009 0.313 -1.242 3.877
C(SENTENCE_JUDGE)[T.Patrick Coughlin]
0.2186 1.074 0.204 0.839 -1.887 2.324
C(SENTENCE_JUDGE)[T.Patrick Foran Lustig]
1.0900 1.503 0.725 0.468 -1.857 4.037
C(SENTENCE_JUDGE)[T.Paul S Pavlus]
0.0493 1.072 0.046 0.963 -2.052 2.151
C(SENTENCE_JUDGE)[T.Paula M Daleo]
-0.2389 1.505 -0.159 0.874 -3.189 2.711
C(SENTENCE_JUDGE)[T.Peggy Chiampas]
0.2074 1.073 0.193 0.847 -1.896 2.310
C(SENTENCE_JUDGE)[T.Ramon Ocasio]
0.1090 1.072 0.102 0.919 -1.993 2.211
C(SENTENCE_JUDGE)[T.Robert Kuzas]
5.915e-17 5.14e-16 0.115 0.908 -9.48e-16 1.07e-15
C(SENTENCE_JUDGE)[T.Ronald S Davis]
0.2182 1.504 0.145 0.885 -2.729 3.165
C(SENTENCE_JUDGE)[T.Ruth Gudino]
0.0832 1.250 0.067 0.947 -2.368 2.534
C(SENTENCE_JUDGE)[T.Samuel J Betar]
0.0643 1.073 0.060 0.952 -2.039 2.168
C(SENTENCE_JUDGE)[T.Sharon Arnold Kanter]
0.5219 1.097 0.476 0.634 -1.629 2.673
C(SENTENCE_JUDGE)[T.Shelley Sutker-Dermer]
0.2377 1.073 0.221 0.825 -1.866 2.342
C(SENTENCE_JUDGE)[T.Sheree D Henry]
-0.0437 1.096 -0.040 0.968 -2.191 2.104
C(SENTENCE_JUDGE)[T.Sophia Atcherson]
0.0767 1.075 0.071 0.943 -2.031 2.184
C(SENTENCE_JUDGE)[T.Stanley Hill]
-0.0215 1.099 -0.020 0.984 -2.175 2.132
C(SENTENCE_JUDGE)[T.Stanley Sacks]
0.2828 1.075 0.263 0.793 -1.824 2.390
C(SENTENCE_JUDGE)[T.Steven G Watkins]
0.4089 1.074 0.381 0.704 -1.697 2.515
C(SENTENCE_JUDGE)[T.Steven J Goebel]
0.1421 1.081 0.131 0.895 -1.978 2.262

```
C(SENTENCE_JUDGE)[T.Steven J Rosenblum]
0.1455      1.071      0.136      0.892      -1.955       2.245
C(SENTENCE_JUDGE)[T.Terence  MacCarthy]
0.2872      1.124      0.256      0.798      -1.916       2.490
C(SENTENCE_JUDGE)[T.Teresa  Molina]
-0.0793      1.107     -0.072      0.943      -2.250       2.091
C(SENTENCE_JUDGE)[T.Terry  Gallagher]
-0.0012      1.072     -0.001      0.999      -2.102       2.100
C(SENTENCE_JUDGE)[T.Thaddeus L Wilson]
0.1887      1.087      0.174      0.862      -1.942       2.319
C(SENTENCE_JUDGE)[T.Thomas J Hennelly]
0.1298      1.083      0.120      0.905      -1.993       2.252
C(SENTENCE_JUDGE)[T.Thomas Paul Panichi]
0.2161      1.504      0.144      0.886      -2.733       3.165
C(SENTENCE_JUDGE)[T.Tiana  Blakely]
-0.0515      1.076     -0.048      0.962      -2.161       2.058
C(SENTENCE_JUDGE)[T.Timothy J Chambers]
0.0811      1.089      0.074      0.941      -2.054       2.217
C(SENTENCE_JUDGE)[T.Timothy Joseph Joyce]
0.5121      1.072      0.478      0.633      -1.590       2.614
C(SENTENCE_JUDGE)[T.Tyria B Walton]
0.2766      1.079      0.256      0.798      -1.838       2.391
C(SENTENCE_JUDGE)[T.URSULA  WALOWSKI]
0.4025      1.072      0.375      0.707      -1.699       2.504
C(SENTENCE_JUDGE)[T.Vincent M Gaughan]
0.7307      1.080      0.677      0.499      -1.387       2.848
C(SENTENCE_JUDGE)[T.Walter  Williams]
0.1696      1.305      0.130      0.897      -2.389       2.728
C(SENTENCE_JUDGE)[T.William  Raines]
0.1360      1.092      0.125      0.901      -2.005       2.277
C(SENTENCE_JUDGE)[T.William B Raines]
0.3816      1.107      0.345      0.730      -1.788       2.551
C(SENTENCE_JUDGE)[T.William G Gamboney]
0.1769      1.076      0.164      0.869      -1.932       2.285
C(SENTENCE_JUDGE)[T.William G Lacy]
-0.6266      1.504     -0.417      0.677      -3.574       2.321
C(SENTENCE_JUDGE)[T.William H Hooks]
0.3979      1.077      0.369      0.712      -1.713       2.509
covid_period_num[T.2]
0.0488      0.052      0.947      0.344      -0.052       0.150
covid_period_num[T.3]
0.1613      0.055      2.945      0.003       0.054       0.269
log_time_delay
0.0196      0.007      2.675      0.007       0.005       0.034
CHARGE_COUNT
-0.0462      0.009     -4.926      0.000      -0.065      -0.028
LENGTH_OF_CASE_in_Days
0.0007   7.28e-05     10.118      0.000       0.001       0.001
CHARGE_COUNT:LENGTH_OF_CASE_in_Days
7.78e-05   1.82e-05      4.265      0.000     4.2e-05       0.000
AGE_AT_INCIDENT
0.0030      0.001      2.946      0.003       0.001       0.005
==============================================================================
==
Omnibus:                    15326.575   Durbin-Watson:                   1.2
42
```

```
Prob(Omnibus):                  0.000   Jarque-Bera (JB):         29528232.0
28
Skew:                          11.676   Prob(JB):                         0.
00
Kurtosis:                     284.087   Cond. No.                     1.67e+
20
==============================================================================
==

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
[2] The smallest eigenvalue is 1.21e-30. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.

Key Metrics and Results:
R-squared: 0.3595
Adjusted R-squared: 0.3448
AIC: 26437.68
BIC: 27863.71

Log(Time Delay) Effect: 0.0196 (p = 0.0075)

Selected Coefficients and P-values:
log_time_delay: Coef = 0.0196, p-value = 0.0075
CHARGE_COUNT: Coef = -0.0462, p-value = 0.0000
AGE_AT_INCIDENT: Coef = 0.0030, p-value = 0.0032
C(RACE_simplified)[T.Black]: Coef = 0.1885, p-value = 0.0667
C(RACE_simplified)[T.Hispanic]: Coef = 0.1288, p-value = 0.2168
C(RACE_simplified)[T.Other]: Coef = -0.1728, p-value = 0.2870
C(GENDER)[T.Male]: Coef = 0.2489, p-value = 0.0000

Note: Detailed coefficients for categorical variables and interactions have
been omitted for brevity.
```

Model Specification and Results Interpretation

We include offense and judge fixed effects to control for unobserved heterogeneity in baseline sentencing severity and docket practices. Judge fixed effects ensure the estimated effect of case delay is identified from within-judge variation rather than cross-judge differences; individual judge coefficients are treated as nuisance controls and not interpreted.

Results show a small but statistically significant positive association between log(time delay) and standardized weighted punishment. Charge count is negatively associated with punishment severity, age at incident and male gender are positively associated, while race effects are imprecisely estimated. High-dimensional fixed effects generate multicollinearity warnings, which are expected in saturated sentencing models and do not affect the interpretation of the main coefficient of interest.

In [36]:
```python
# Access the design matrix (exogenous variables) from the fitted model
X = results.model.exog  # This accesses the design matrix of predictors
```

```
variable_names = results.model.exog_names  # This accesses the names of the

# Calculate Variance Inflation Factor
vif_data = pd.DataFrame({
    "Variable": variable_names,
    "VIF": [variance_inflation_factor(X, i) for i in range(X.shape[1])]
})

print(vif_data)
```

```
/Users/tyf981126/Library/Python/3.11/lib/python/site-packages/statsmodels/regression/linear_model.py:1782: RuntimeWarning: invalid value encountered in scalar divide
  return 1 - self.ssr/self.centered_tss
                                                Variable          VIF
0                                              Intercept  9559.300185
1    C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated Battery]    12.885714
2    C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated Batte...     9.119060
3    C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated Batte...     2.145188
4        C(UPDATED_OFFENSE_CATEGORY)[T.Aggravated DUI]     6.220561
..                                                 ...          ...
204                                     log_time_delay     1.230460
205                                       CHARGE_COUNT     7.725195
206                              LENGTH_OF_CASE_in_Days     2.098203
207                CHARGE_COUNT:LENGTH_OF_CASE_in_Days     8.424920
208                                     AGE_AT_INCIDENT     1.181988

[209 rows x 2 columns]
```

Multicollinearity check (VIF)

VIF values are high for the intercept and several categorical indicators, which is expected in models with many fixed effects (offense and judge dummies). In contrast, the main variables of interest—log(time delay), age at incident, and case length—have low VIFs, indicating limited multicollinearity for key regressors. Overall, multicollinearity primarily affects nuisance fixed effects rather than the identification of the main delay effect.

In [37]:
```
# Data for the summary table with revised variable names
data = {
    "Variable": [
        "Intercept",
        "Log(Time Delay)",
        "Charge Count",
        "Length of Case (Days)",
        "Case Complexity",
        "Age at Incident",
        "COVID Period (Post-COVID)",
        "Race (Black)",
        "Gender (Male)"
    ],
    "Coefficient": [
        0.2578, 0.0196,-0.0462,0.0007,7.78e-05,0.0030,0.1613,0.1885,0.2489],
```

```
    "Std. Error": [0.019,0.007,0.009,7.28e-05,1.82e-05,0.001,0.055,0.103,0.0
    ],
    "t-Statistic": [13.589,2.675,-4.926,10.118,4.265,2.946,2.945,1.834,7.244
    ],
    "P-value": [0.000,0.007,0.000,0.000,0.000,0.003,0.003,0.067,0.000
    ]
}

# Create the DataFrame
summary_table = pd.DataFrame(data)

# Format numeric columns for better readability
summary_table["Coefficient"] = summary_table["Coefficient"].apply(lambda x:
summary_table["Std. Error"] = summary_table["Std. Error"].apply(lambda x: f"
summary_table["t-Statistic"] = summary_table["t-Statistic"].apply(lambda x:
summary_table["P-value"] = summary_table["P-value"].apply(lambda x: f"{x:.4f


summary_table
```

Out[37]:

	Variable	Coefficient	Std. Error	t-Statistic	P-value
0	Intercept	0.2578	0.0190	13.589	0.0000
1	Log(Time Delay)	0.0196	0.0070	2.675	0.0070
2	Charge Count	-0.0462	0.0090	-4.926	0.0000
3	Length of Case (Days)	0.0007	0.0001	10.118	0.0000
4	Case Complexity	0.0001	0.0000	4.265	0.0000
5	Age at Incident	0.0030	0.0010	2.946	0.0030
6	COVID Period (Post-COVID)	0.1613	0.0550	2.945	0.0030
7	Race (Black)	0.1885	0.1030	1.834	0.0670
8	Gender (Male)	0.2489	0.0340	7.244	0.0000

A 1% increase in the log-transformed time delay is associated with a 0.0198-unit increase in the standardized weighted punishment. This indicates that longer delays are linked to slightly higher punishments, even after controlling for other factors.

Each additional charge is associated with a 0.0097-unit decrease in the standardized weighted punishment. This could suggest that cases with more charges involve less severe individual charges, which reduces the overall punishment severity.

Each additional year in the defendant's age at the time of the incident is associated with a 0.0031-unit increase in the standardized weighted punishment. This suggests older defendants may face slightly harsher punishments.

Black defendants receive standardized weighted punishments 0.187 units higher than the reference category (e.g., Asian or other baseline groups). The result is marginally significant, suggesting a potential disparity in punishment severity.

Bisecting K-means

In [38]:
```
# Extract unique crime types from the dataset
unique_crime_types = df_delayed['UPDATED_OFFENSE_CATEGORY'].unique()

# Display the unique values for review
unique_crime_types
```

Out[38]:
```
array(['Other Offense', 'Aggravated Battery Police Officer',
       'Aggravated Robbery', 'Burglary', 'Attempt Armed Robbery',
       'Fraudulent ID', 'Domestic Battery',
       'Reckless Discharge of Firearm', 'Aggravated Battery',
       'Criminal Damage to Property', 'Escape - Failure to Return',
       'Attempt Homicide', 'Identity Theft', 'Theft', 'Armed Robbery',
       'Home Invasion', 'Robbery', 'Aggravated Fleeing and Eluding',
       'UUW - Unlawful Use of Weapon', 'Residential Burglary',
       'Retail Theft', 'Homicide', 'Sex Crimes', 'Aggravated DUI',
       'Narcotics', 'Vehicular Hijacking',
       'Possession of Stolen Motor Vehicle', 'Obstructing Justice',
       'Failure to Register as a Sex Offender',
       'Violation Order Of Protection', 'Forgery', 'Child Pornography',
       'Aggravated Battery With A Firearm', 'Credit Card Cases',
       'Kidnapping', 'Intimidation', 'Aggravated Assault Police Officer',
       'Child Abduction', 'Aggravated Discharge Firearm', 'Stalking',
       'Driving With Suspended Or Revoked License',
       'Criminal Trespass To Residence', 'Vehicular Invasion',
       'Reckless Homicide', 'Gun - Non UUW', 'Arson',
       'Impersonating Police Officer', 'Hate Crimes', 'Major Accidents',
       'Attempt Arson', 'Possession of Contraband in Penal Institution',
       'Official Misconduct', 'Fraud', 'Prostitution',
       'Possession of Explosives', 'Human Trafficking',
       'Deceptive Practice', 'Unlawful Restraint',
       'Attempt Vehicular Hijacking', 'Bribery', 'Bomb Threat',
       'Communicating With Witness', 'Disarming Police Officer',
       'Tampering'], dtype=object)
```

Group crime types for clustering

In [39]:
```
# Mapping crime types to categories
crime_group_mapping = {
    'Aggravated Battery Police Officer': 'Violent Crimes',
    'Aggravated Battery': 'Violent Crimes',
    'Aggravated Battery With A Firearm': 'Violent Crimes',
    'Aggravated Assault Police Officer': 'Violent Crimes',
    'Domestic Battery': 'Violent Crimes',
    'Attempt Homicide': 'Violent Crimes',
    'Reckless Discharge of Firearm': 'Violent Crimes',
    'Aggravated Discharge Firearm': 'Violent Crimes',
    'Homicide': 'Violent Crimes',
    'Kidnapping': 'Violent Crimes',
    'Stalking': 'Violent Crimes',
    'Robbery': 'Violent Crimes',
    'Armed Robbery': 'Violent Crimes',
```

```
    'Aggravated Robbery': 'Violent Crimes',
    'Home Invasion': 'Violent Crimes',
    'Burglary': 'Property Crimes',
    'Theft': 'Property Crimes',
    'Identity Theft': 'Property Crimes',
    'Residential Burglary': 'Property Crimes',
    'Retail Theft': 'Property Crimes',
    'Forgery': 'Property Crimes',
    'Fraudulent ID': 'Property Crimes',
    'Credit Card Cases': 'Property Crimes',
    'Criminal Damage to Property': 'Property Crimes',
    'Possession of Stolen Motor Vehicle': 'Property Crimes',
    'Deceptive Practice': 'Property Crimes',
    'Narcotics': 'Drug-Related Crimes',
    'Possession of Contraband in Penal Institution': 'Drug-Related Crimes',
    'Aggravated DUI': 'Drug-Related Crimes',
    'Sex Crimes': 'Sexual and Exploitation Crimes',
    'Child Pornography': 'Sexual and Exploitation Crimes',
    'Human Trafficking': 'Sexual and Exploitation Crimes',
    'Failure to Register as a Sex Offender': 'Sexual and Exploitation Crimes
    'Prostitution': 'Sexual and Exploitation Crimes',
    'Violation Order Of Protection': 'Public Order and Regulatory Crimes',
    'Driving With Suspended Or Revoked License': 'Public Order and Regulator
    'Reckless Homicide': 'Public Order and Regulatory Crimes',
    'Gun - Non UUW': 'Public Order and Regulatory Crimes',
    'Arson': 'Public Order and Regulatory Crimes',
    'Possession of Explosives': 'Public Order and Regulatory Crimes',
    'Official Misconduct': 'Public Order and Regulatory Crimes',
    'Unlawful Restraint': 'Public Order and Regulatory Crimes',
    'Tampering': 'Public Order and Regulatory Crimes',
    'Other Offense': 'Miscellaneous Crimes',
    'Impersonating Police Officer': 'Miscellaneous Crimes',
    'Hate Crimes': 'Miscellaneous Crimes',
    'Major Accidents': 'Miscellaneous Crimes',
    'Attempt Arson': 'Miscellaneous Crimes',
    'Attempt Vehicular Hijacking': 'Miscellaneous Crimes',
    'Vehicular Hijacking': 'Miscellaneous Crimes',
    'Communicating With Witness': 'Miscellaneous Crimes',
    'Bribery': 'Miscellaneous Crimes',
    'Bomb Threat': 'Miscellaneous Crimes',
    'Disarming Police Officer': 'Miscellaneous Crimes'
}

# Apply the mapping to the dataset
df_delayed['Crime_Group'] = df_delayed['UPDATED_OFFENSE_CATEGORY'].map(crime
```

In [40]:
```
# Ensure necessary features are selected for clustering, including the crime
features = ['log_time_delay', 'weighted_punishment_std', 'CHARGE_COUNT',
            'AGE_AT_INCIDENT', 'LENGTH_OF_CASE_in_Days', 'GENDER', 'Crime_Gr

# Subset, drop NaN values, and filter for valid gender values
df_clustering = df_delayed[features].dropna()
df_clustering = df_clustering[df_clustering['GENDER'].isin(['Female', 'Male'

# Dummy encode the Crime_Group and GENDER variables
crime_dummies = pd.get_dummies(df_clustering['Crime_Group'], drop_first=True
```

```
gender_dummies = pd.get_dummies(df_clustering['GENDER'], drop_first=True)  #

# Combine the numeric features with the dummy variables
df_clustering_combined = pd.concat([
    df_clustering.drop(['Crime_Group', 'GENDER'], axis=1),
    crime_dummies,
    gender_dummies
], axis=1)

# Scale the combined dataset
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_clustering_combined)
```

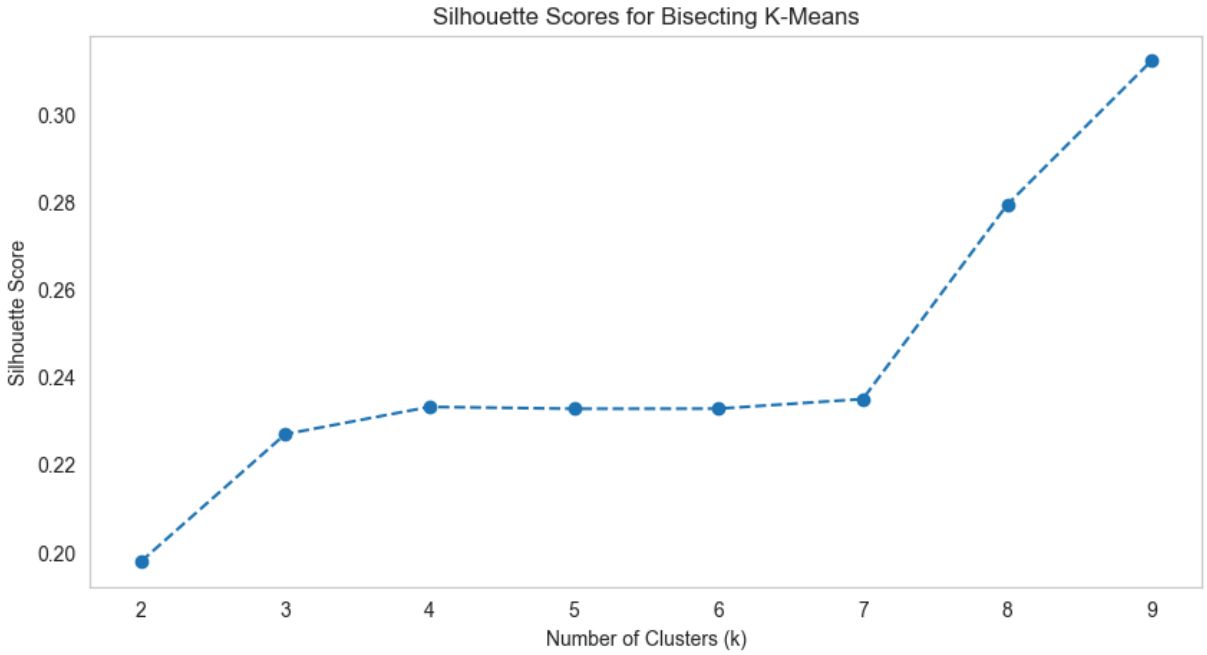
Bisecting definition function

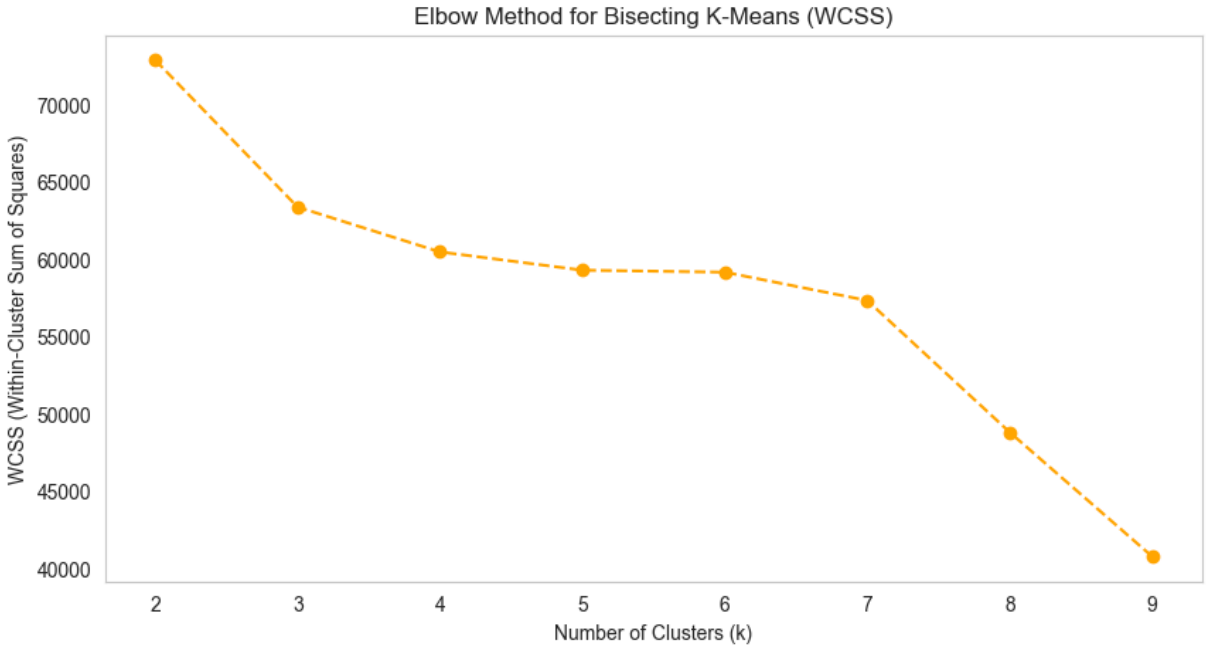
In [41]:
```
def bisecting_kmeans(data, max_clusters=5, random_state=42):
    clusters = [data]
    labels = np.zeros(len(data))
    current_cluster = 0

    for i in range(max_clusters - 1):
        # Select the cluster to split (largest variance or largest size)
        current_data = clusters[current_cluster]

        # Check for small clusters
        if len(current_data) < 2:
            continue

        # Apply k-means with k=2
        kmeans = KMeans(n_clusters=2, random_state=random_state)
        kmeans.fit(current_data)
        new_labels = kmeans.labels_

        # Update clusters and labels
        cluster_0 = current_data[new_labels == 0]
        cluster_1 = current_data[new_labels == 1]

        clusters[current_cluster] = cluster_0
        clusters.append(cluster_1)

        # Update global labels
        labels[np.where(labels == current_cluster)[0][new_labels == 1]] = le

        # Select next cluster to split (based on variance)
        current_cluster = np.argmax([np.var(cluster, axis=0).sum() for clust


    return labels
```

Find Optimal k(separate lines)

In [44]:

```python
# Define a range of k values to test
k_values = range(2, 10)
silhouette_scores = []
wcss_values = []

# Calculate silhouette scores and WCSS for each k
for k in k_values:
    labels = bisecting_kmeans(scaled_data, max_clusters=k)
    silhouette_avg = silhouette_score(scaled_data, labels)
    silhouette_scores.append(silhouette_avg)

    # Compute WCSS (Within-Cluster Sum of Squares)
    wcss = 0
    for cluster in set(labels):
        cluster_points = scaled_data[labels == cluster]
        cluster_center = cluster_points.mean(axis=0)
        wcss += ((cluster_points - cluster_center) ** 2).sum()
    wcss_values.append(wcss)

# Plot Silhouette Scores
plt.figure(figsize=(10, 5))
plt.plot(k_values, silhouette_scores, marker='o', linestyle='--')
plt.title("Silhouette Scores for Bisecting K-Means")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Silhouette Score")
plt.grid()
plt.show()

# Plot WCSS
plt.figure(figsize=(10, 5))
plt.plot(k_values, wcss_values, marker='o', linestyle='--', color='orange')
plt.title("Elbow Method for Bisecting K-Means (WCSS)")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("WCSS (Within-Cluster Sum of Squares)")
plt.grid()
plt.show()
```

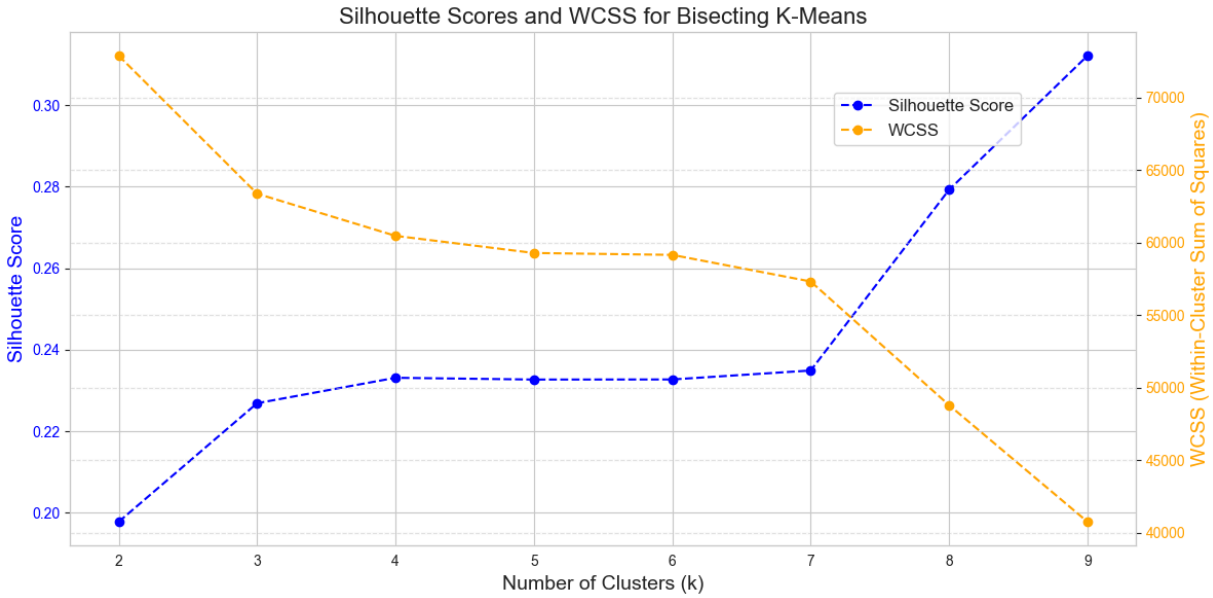




In one plot

In [45]:
```
# Combine Silhouette Scores and WCSS in one plot
fig, ax1 = plt.subplots(figsize=(12, 6))

# Plot Silhouette Scores on the primary y-axis
ax1.plot(k_values, silhouette_scores, marker='o', linestyle='--', label='Sil
ax1.set_xlabel("Number of Clusters (k)", fontsize=14)
ax1.set_ylabel("Silhouette Score", fontsize=14, color='blue')
ax1.tick_params(axis='y', labelcolor='blue')
ax1.set_title("Silhouette Scores and WCSS for Bisecting K-Means", fontsize=1

# Create a second y-axis for WCSS
ax2 = ax1.twinx()
ax2.plot(k_values, wcss_values, marker='o', linestyle='--', label='WCSS', co
```

```
ax2.set_ylabel("WCSS (Within-Cluster Sum of Squares)", fontsize=14, color='c
ax2.tick_params(axis='y', labelcolor='orange')

# Add a legend
fig.legend(loc="upper right", bbox_to_anchor=(0.9, 0.9), bbox_transform=ax1.

# Show grid for better visualization
plt.grid(visible=True, which='major', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```



Insights:

Silhouette scores increase gradually with the number of clusters, while WCSS declines smoothly, indicating improved fit but diminishing returns beyond mid-range values of k. The absence of a sharp elbow suggests no single “natural” partition; instead, moderate cluster counts (e.g., k = 4–6) offer a balance between interpretability and cohesion. This aligns with the criminal sentencing context, where heterogeneity in case characteristics is continuous rather than discretely separable, and clustering is used primarily for descriptive stratification rather than definitive classification.

In [46]:
```
# Run bisecting k-means
cluster_labels = bisecting_kmeans(scaled_data, max_clusters=4)

# Add cluster labels to the original dataset
df_clustering_combined['Cluster'] = cluster_labels

# Evaluate clustering
silhouette_avg = silhouette_score(scaled_data, cluster_labels)

# Summarize cluster counts
cluster_counts = df_clustering_combined['Cluster'].value_counts()

# Summarize mean values of features by cluster
```

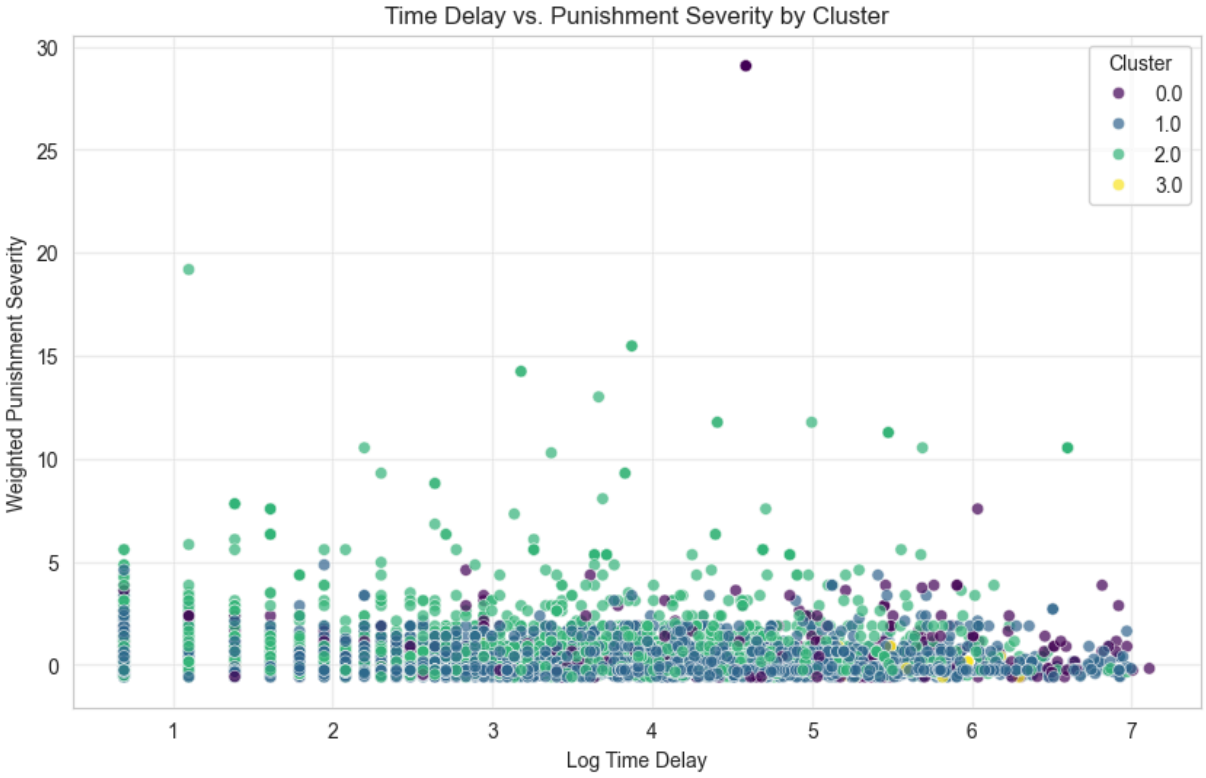
```
cluster_means = df_clustering_combined.groupby('Cluster').mean()

# Display cluster counts and means
summary = {"Cluster Counts": cluster_counts, "Cluster Means": cluster_means}
summary
```

Out[46]:

```
{'Cluster Counts': Cluster
 1.0    4531
 2.0    2594
 0.0     528
 3.0      51
 Name: count, dtype: int64,
 'Cluster Means':          log_time_delay  weighted_punishment_std  CHARGE_
COUNT  \
 Cluster
 0.0            3.974907                 0.589607      3.070076
 1.0            3.386188                -0.008798      1.352019
 2.0            2.927737                 0.644858      2.402467
 3.0            4.262550                 0.205579     30.647059

          AGE_AT_INCIDENT  LENGTH_OF_CASE_in_Days  Miscellaneous Crimes  \
 Cluster
 0.0            36.094697              435.689394              0.000000
 1.0            35.431031              283.056500              0.108585
 2.0            31.253277              401.309946              0.002313
 3.0            35.509804              406.078431              0.000000

          Property Crimes  Public Order and Regulatory Crimes  \
 Cluster
 0.0             0.000000                            0.000000
 1.0             0.667623                            0.048113
 2.0             0.000386                            0.000000
 3.0             0.000000                            0.000000

          Sexual and Exploitation Crimes  Violent Crimes      Male
 Cluster
 0.0                            1.000000        0.000000  0.973485
 1.0                            0.000000        0.000000  0.831163
 2.0                            0.000000        0.970316  0.868157
 3.0                            0.980392        0.000000  0.960784  }
```

In [47]:

```
# Clustering
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_clustering_combined, x='log_time_delay', y='weighted
                hue='Cluster', palette='viridis', alpha=0.7)
plt.title("Time Delay vs. Punishment Severity by Cluster")
plt.xlabel("Log Time Delay")
plt.ylabel("Weighted Punishment Severity")
plt.grid(alpha=0.3)
plt.show()
```

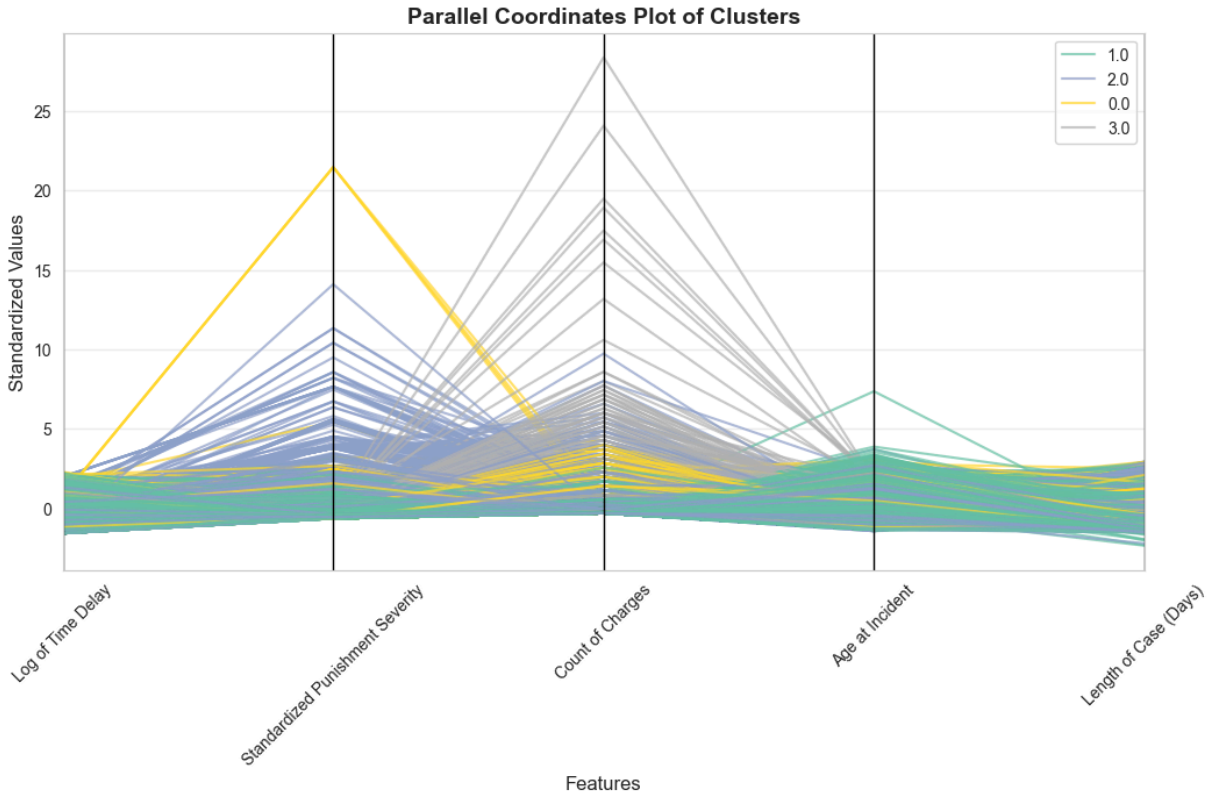


Visualization note.

While clustering highlights heterogeneity in the joint distribution of case delay and punishment severity, this scatter plot suffers from substantial overplotting and cluster overlap, particularly at low punishment levels where most observations concentrate. As a result, cluster boundaries are not visually distinct, and the figure adds limited interpretive value beyond the regression analysis. We therefore do not include this visualization in the main report and use clustering only as a descriptive robustness check rather than a primary analytical tool.

In [48]:
```
# Parallel coordinates plot
# Define a dictionary to map column names to more readable labels
variable_name_mapping = {
    'log_time_delay': 'Log of Time Delay',
    'weighted_punishment_std': 'Standardized Punishment Severity',
    'CHARGE_COUNT': 'Count of Charges',
    'AGE_AT_INCIDENT': 'Age at Incident',
    'LENGTH_OF_CASE_in_Days': 'Length of Case (Days)'
}

# Select relevant features and include Cluster
features = ['log_time_delay', 'weighted_punishment_std', 'CHARGE_COUNT', 'AG
parallel_data = df_clustering_combined[features + ['Cluster']].copy()

# Ensure Cluster is categorical for the plot
parallel_data['Cluster'] = parallel_data['Cluster'].astype(str)

# Standardize only numerical features
scaler = StandardScaler()
```

```
numerical_features = features
parallel_data[numerical_features] = scaler.fit_transform(parallel_data[numer

# Plot parallel coordinates
plt.figure(figsize=(12, 6))
parallel_coordinates(parallel_data, 'Cluster', colormap='Set2', alpha=0.7)

# Map the original column names to the new friendly names for the x-axis
current_labels = parallel_data.columns[:-1]  # Exclude 'Cluster'
new_labels = [variable_name_mapping[label] for label in current_labels]
plt.gca().set_xticks(range(len(current_labels)))
plt.gca().set_xticklabels(new_labels, rotation=45, fontsize=10)

# Add titles and labels
plt.title('Parallel Coordinates Plot of Clusters', fontsize=14, fontweight='
plt.xlabel('Features', fontsize=12)
plt.ylabel('Standardized Values', fontsize=12)
plt.grid(alpha=0.3)
plt.show()
```



- Cluster 2 is characterized by higher values on the charge-count dimension relative to other clusters.
- This separation suggests that charge count is a salient feature distinguishing cases with higher punishment severity, consistent with regression results identifying charge-related measures as important correlates.

Random Forest Model

In [50]:
```
# Select features and target
categorical_features = ['RACE_simplified','Crime_Group']
numerical_features = ['log_time_delay', 'CHARGE_COUNT', 'AGE_AT_INCIDENT', '
target = 'weighted_punishment_std_category'

# Encode Gender as Male = 1, Female = 0, and handle missing/unknown values
df_delayed['GENDER'] = df_delayed['GENDER'].map({'Male': 1, 'Female': 0}).fi

# One-hot encode categorical features
encoder = OneHotEncoder(drop='first', sparse_output=False)
encoded_categorical = pd.DataFrame(
    encoder.fit_transform(df_delayed[categorical_features]),
    columns=encoder.get_feature_names_out(categorical_features),
    index=df_delayed.index
)


scaler = StandardScaler()
scaled_numerical = pd.DataFrame(
    scaler.fit_transform(df_delayed[numerical_features]),
    columns=numerical_features,
    index=df_delayed.index
)

# Combine numerical and categorical features
features_combined = pd.concat([scaled_numerical, encoded_categorical], axis=

# Prepare target variable (binary classification based on median)
df_delayed[target] = (df_delayed['weighted_punishment_std'] > df_delayed['we

# Check target variable distribution
print("Target variable distribution:")
print(df_delayed[target].value_counts(normalize=True))
```

```
Target variable distribution:
weighted_punishment_std_category
0    0.519268
1    0.480732
Name: proportion, dtype: float64
```

In [51]:
```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    features_combined, df_delayed[target], test_size=0.3, random_state=42
)

# Initialize the model
rf_model = RandomForestClassifier(random_state=42)

from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

rng = np.random.default_rng(42)
```

```python
# Parameter distributions for random search
param_distributions = {
    # Number of trees in the forest - uniform integer distribution
    'n_estimators': randint(100, 300),

    # Maximum depth of the tree - adding None as a special case
    'max_depth': [None, *rng.integers(5, 50, size=4)],  # Will include None

    # Number of features to consider
    'max_features': ['sqrt', 'log2', None, 0.5],  # Keeping discrete choices

    # Minimum samples for split - uniform integer distribution
    'min_samples_split': randint(2, 10),

    # Minimum samples for leaf - uniform integer distribution
    'min_samples_leaf': randint(1, 10),

    # Bootstrap - keeping binary choice
    'bootstrap': [True, False],

    # Class weight - keeping discrete choices
    'class_weight': [None, 'balanced', 'balanced_subsample']
}

# Configure Random Search
random_search = RandomizedSearchCV(
    estimator=rf_model,
    param_distributions=param_distributions,
    n_iter=300,
    scoring='accuracy',
    cv=5,
    n_jobs=-1,
    verbose=1
)
# Fit Grid Search
random_search.fit(X_train, y_train)

# Best parameters
print("Best Hyperparameters:", random_search.best_params_)

# Best model
best_rf_model = random_search.best_estimator_

# Evaluate on test set
y_pred = best_rf_model.predict(X_test)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Feature importance
feature_importances = pd.DataFrame({
    'Feature': features_combined.columns,
    'Importance': best_rf_model.feature_importances_
}).sort_values(by='Importance', ascending=False)

# Display results
print("\nTop 10 Most Important Features:")
```

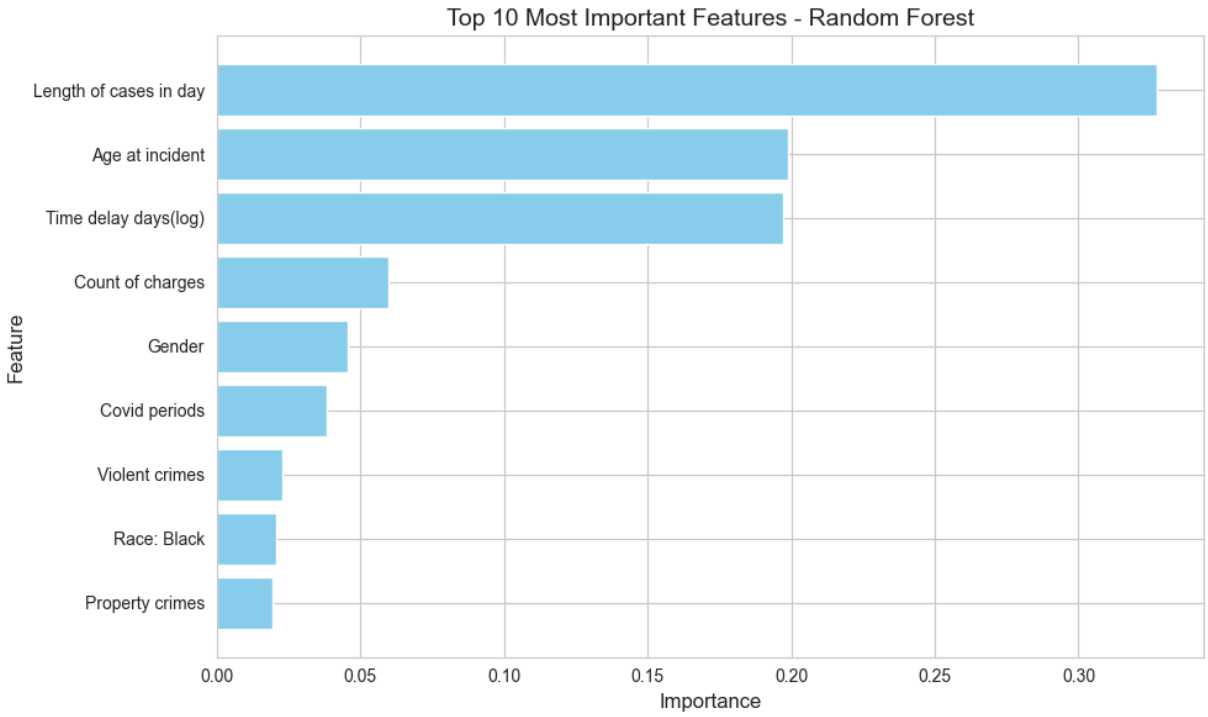
```
print(feature_importances.head(10))

# Display confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(conf_matrix)
```

```
Fitting 5 folds for each of 300 candidates, totalling 1500 fits
Best Hyperparameters: {'bootstrap': True, 'class_weight': 'balanced_subsampl
e', 'max_depth': np.int64(34), 'max_features': 'log2', 'min_samples_leaf':
1, 'min_samples_split': 4, 'n_estimators': 151}

Classification Report:
              precision    recall  f1-score   support

           0       0.73      0.70      0.72      1476
           1       0.68      0.71      0.69      1311

    accuracy                           0.71      2787
   macro avg       0.70      0.71      0.70      2787
weighted avg       0.71      0.71      0.71      2787


Top 10 Most Important Features:
                         Feature  Importance
3        LENGTH_OF_CASE_in_Days    0.327204
2                AGE_AT_INCIDENT    0.198721
0                 log_time_delay    0.197083
1                   CHARGE_COUNT    0.059896
4                         GENDER    0.045595
5               covid_period_num    0.038396
14   Crime_Group_Violent Crimes    0.023158
6          RACE_simplified_Black    0.020786
11  Crime_Group_Property Crimes    0.019335
15              Crime_Group_nan    0.013655

Confusion Matrix:
[[1037  439]
 [ 383  928]]
```

Accuracy: The model achieves an overall accuracy of 71%, indicating that 71% of the predictions match the actual labels in the test set. This is a moderate level of performance, suggesting the model has room for improvement but performs better than random guessing.

In [53]:
```
# Rename features for better readability
top_features = feature_importances.head(9).copy()
top_features['Feature'] = top_features['Feature'].replace({
    'LENGTH_OF_CASE_in_Days': 'Length of cases in day',
    'AGE_AT_INCIDENT': 'Age at incident',
    'CHARGE_COUNT': 'Count of charges',
    'covid_period_num': 'Covid periods',
    'Crime_Group_Violent Crimes':'Violent crimes',
    'GENDER': 'Gender',
    'Crime_Group_Property Crimes':'Property crimes',
```

```
    'RACE_simplified_Black': 'Race: Black',
    'RACE_simplified_White': 'Race: White',
    'RACE_simplified_Hispanic': 'Race: Hispanic',
    'Crime_Group_Sexual and Exploitation Crimes':'Sexual and exploitation cr
    'RACE_simplified_Other': 'Race: Other',
    'log_time_delay': 'Time delay days(log)',
    'GENDER': 'Gender'
})

# Plot the updated feature importances
plt.figure(figsize=(10, 6))
plt.barh(top_features['Feature'], top_features['Importance'], color='skyblue
plt.xlabel('Importance', fontsize=12)
plt.ylabel('Feature', fontsize=12)
plt.title('Top 10 Most Important Features - Random Forest', fontsize=14)
plt.gca().invert_yaxis()  # Highest importance at the top
plt.tight_layout()
plt.show()
```



Feature importance (Random Forest).

Case length, age at incident, and log time delay emerge as the most influential predictors of punishment severity in the Random Forest model. Charge count and gender contribute modestly, while crime type indicators and race-related variables have comparatively lower importance. These results reinforce the regression findings that temporal and case-processing factors play a central role in explaining punishment severity, while demographic and categorical factors contribute less to predictive power.